Contents

Template 3
- Template 3
- templatePy 3

Algebra 4
- Berlekamp Massey 4
- mod inv 4
- recurrencias 5

Bits 5
- Bits operations 5

Number Theory 6
- Extended gcd 6
- Inverse mod 7
- Is prime for large numbers 7
- Modulo long long 7
- NTT 7
- Prime factorization 8

Data Structures 8
- AVL 8
- Bitset 11
- Deque 11
- DSU 12
- List 13
- MinMax Heap 14
- Monotonic MeteSaca 14
- Ordered Set 15
- Priority Queue 16
- LazySegmentTree 17
  - Assing On A Range And Look On Position 17
  - Sum And Update On Range 18
  - Sum On A Range And Minimum 19
- MergeSortTree 20
  - MergeSortTree 20
- SegmentTree 21
  - 2D Segment Tree 21
  - Segment Tree 22

Graphs 23
- Flows 23
  - Closure Problem 23
  - Flow 24
  - Min Cost Max Flow (Dinic) 25
  - Min Cost Max Flow (Edmond Karps) 26
  - Min Cut 27
  - Push Relabel Flow 27
- Graphs algorthms 28
  - Acyclic Ordering 28
  - Bellman-Ford 29
  - Cycle Detection Directed 31
  - Cycle Detection Undirected 31
  - Dijkstra 32
  - Euler tour 32
  - Floyd Warshall 33
  - Fuertemente Conexa Kosaraju-Sharir 34
  - Havel-hakin 35
  - Is Bipartite 35
  - Tarjan 35
  - Tarjan puentes 36
  - Topological Sort 37
  - Weighted Bipartite Matching (Hungarian) 37
- Trees 38
  - Centroid Decomposition 38
  - Farthest node for each node 39
  - Heavy light decomposition 39
  - LCA 40

Strings 41
- Aho-Corasik 41
- Counting Substrings 43
- Finding Borders 44
- Finding Periods 44
- Hashing 45
- Hashing Strings as vectors 46
- KMP 46
- Longest Common Substring 47
- Manacher 47
- Number of Balanced Substrings 48
- Number of Distinct Substrings 48
- Occurrences Of Each Prefixes 49
- Prefix Function 49
- Repeating Substring 49

Suffix Array and LCP . . . 50
Trie . . . 51
Z function . . . 52

Dynamic Programming **52**
Coin Combinations I . . . 52
Coin Combinations II . . . 53
Digit DP . . . 53
Edit Distance . . . 54
Knapsack 01 . . . 54
Knapsack 01 optimized . . . 54
Knapsack 2 . . . 55
Knapsack 2 optimized . . . 55
Longest Common Subsecuence . . . 55
Longest Increasing Subsecuence . . . 56

Geometry **56**
Closest Pair . . . 56
Complex Points . . . 57
Convex Hull . . . 58
Delaunay . . . 58
KD Tree . . . 60
Li Chao Tree . . . 62
Line . . . 62
Minimum Enclosing Circle . . . 63
Online Convex Hull . . . 64
Picks Theorem . . . 65
Points . . . 66
Shoelace Formula . . . 66

Others **67**
All Submask . . . 67
Binpow . . . 67
Coordinate Compression . . . 67
Cover Points By Intervals . . . 67
FFT . . . 68
i128 . . . 69
Kadanes . . . 70
Matrix . . . 70
Matrix Exponentiation . . . 71
Matrix with array . . . 71
Modular binpow . . . 72
Preifix Sum 2D . . . 72
Static Range Minimum Query . . . 72
Ternary search . . . 73

Template

Template

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

#define all(v) v.begin(), v.end()
#define rall(v) v.rbegin(), v.rend()
#define pb push_back
#define fi first
#define se second
#define sz(a) int(a.size())
#define endl "\n"
using ll = long long;
using vi = vector<int>;
using vll = vector<ll>;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
template <class T>
using oset = tree<T, null_type, less<T>,
                  rb_tree_tag, tree_order_statistics_node_update>;

ll gcd(ll a, ll b) {return a == 0? b: gcd(b%a,a);}
ll lcm(ll a, ll b) {return a * (b / gcd(a,b));}

int tc;

#define error(...) dbgh(#__VA_ARGS__, __VA_ARGS__)

template <typename T>
void dbgh(const char *names, T value)
{
  cerr << names << " = " << value << endl;
}

template <typename T, typename... Args>
void dbgh(const char *names, T value, Args... args)
{
  const char *comma = strchr(names, ',');
  cerr.write(names, comma - names) << " = " << value << endl;
  dbgh(comma + 1, args...);
}

constexpr ll INF_LL = LONG_LONG_MAX;
constexpr int INF_INT = INT_MAX;

void solve() {}

int main()
{
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int t = 1;
  // cin >> t;
  for (tc = 1; tc <= t; tc++)
    solve();
  return 0;
}
```

templatePy

```python
import sys

def main():
    data = sys.stdin.read().strip().split()
    numbers = list(map(int, data))

    output = []
    for num in numbers:
        output.append(f"Número leído: {num}")

    sys.stdout.write("\n".join(output))

if __name__ == '__main__':
    main()
```

Algebra

Berlekamp Massey

```
/**
 * Description: Recovers a nth-order linear recurrence relation from
 ↪  the first
 *   2n terms. Solve can be sped up to O(n log n log k) with FFT as
 ↪  mul.
 * Source: https://codeforces.com/blog/entry/61306,
 *   https://github.com/kth-competitive-programming/kactl
 * Verification: https://codeforces.com/contest/506/problem/E
 * Time: O(n^2) BM, O(n^2 log k) solve
 */

template<typename T>
vector<T> berlekampMassey(const vector<T> &s) {
    int n = (int) s.size(), l = 0, m = 1;
    vector<T> b(n), c(n);
    T ld = b[0] = c[0] = 1;
    for (int i=0; i<n; i++, m++) {
        T d = s[i];
        for (int j=1; j<=l; j++)
            d += c[j] * s[i-j];
        if (d == 0)
            continue;
        vector<T> temp = c;
        T coef = d / ld;
        for (int j=m; j<n; j++)
            c[j] -= coef * b[j-m];
        if (2 * l <= i) {
            l = i + 1 - l;
            b = temp;
            ld = d;
            m = 0;
        }
    }
    c.resize(l + 1);
    c.erase(c.begin());
    for (T &x : c)
        x = -x;
    return c;
}

template<typename T>
T solve(const vector<T> &c, const vector<T> &s, long long k) {
    int n = (int) c.size();
    assert(c.size() <= s.size());

    auto mul = [&] (const vector<T> &a, const vector<T> &b) ->
    ↪  vector<T> {
        vector<T> ret(a.size() + b.size() - 1);
        for (int i=0; i<(int)a.size(); i++)
            for (int j=0; j<(int)b.size(); j++)
                ret[i+j] += a[i] * b[j];
        for (int i=(int)ret.size()-1; i>=n; i--)
            for (int j=n-1; j>=0; j--)
                ret[i-j-1] += ret[i] * c[j];
        ret.resize(min((int) ret.size(), n));
        return ret;
    };

    vector<T> a = n == 1 ? vector<T>{c[0]} : vector<T>{0, 1}, x{1};
    for (; k>0; k/=2) {
        if (k % 2)
            x = mul(x, a);
        a = mul(a, a);
    }
    x.resize(n);

    T ret = 0;
    for (int i=0; i<n; i++)
        ret += x[i] * s[i];
    return ret;
}
```

mod inv

```
#include <iostream>
using namespace std;

int euclides_extendido(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
```

```
    }
    int x1, y1;
    int d = euclides_extendido(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return d;
}

// Devuelve el inverso modular de a módulo m, o -1 si no existe
int inverso_modular(int a, int m) {
    int x, y;
    int g = euclides_extendido(a, m, x, y);
    if (g != 1) return -1; // No existe inverso si a y m no son
    ↪ coprimos
    x = (x % m + m) % m;   // Asegura que x sea positivo
    return x;
}
```

recurrencias

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
typedef long long ll;
ll sn, sn_1;

vector<vector<ll>> multiply(const vector<vector<ll>>& A, const
↪ vector<vector<ll>>& B) {
    int N = A.size();
    vector<vector<ll>> C(N, vector<ll>(N, 0));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < N; k++) {
                ll r = A[i][k] * B[k][j] % MOD;
                C[i][j] = (C[i][j] + r) % MOD;
            }
        }
    }
    return C;
}
vector<vector<ll>> matrixExpo(vector<vector<ll>> T, ll p) {
    int N = T.size();
    vector<vector<ll>> res(N, vector<ll>(N, 0));
    for (int i = 0; i < N; i++) res[i][i] = 1;
    while (p > 0) {
        if (p & 1)
            res = multiply(res, T);
        T = multiply(T, T);
        p >>= 1;
    }
    return res;
}

void calculateFib(int sk){
    vector<vector<ll>> F = {{1, 1},
                            {1, 0}};
    vector<vector<ll>> base = {{sn},
                               {sn_1}};
    vector<vector<ll>> res = multiply(matrixExpo(F, sk), base);
    sn= res[0][0];
    sn_1 = res[1][0];
}
```

Bits

Bits operations

```
// Ver si n es divisible por 2^k
bool isDivisibleByPowerOf2(int n, int k) {
  int powerOf2 = 1 << k;
  return (n & (powerOf2 - 1)) == 0;
}

bool isPowerOfTwo(unsigned int n) {
  return n && !(n & (n - 1));
}

int countSetBits(int n)
{
```

```
  int count = 0;
  while (n)
  {
      n = n & (n - 1);
      count++;
  }
  return count;
}

// Cuenta el número de bits encendidos hasta el número n
int countSetBits(int n) {
  int count = 0;
  while (n > 0) {
      int x = std::bit_width(n) - 1;
      count += x << (x - 1);
      n -= 1 << x;
      count += n + 1;
  }
  return count;
}

// Encender el i-ésimo bit
n |= (1 << i);

// Apagar el i-ésimo bit
n &= ~(1 << i);

// Alternar el i-ésimo bit
n ^= (1 << i);

// Apagar el bits más a la derecha
n = n & (n-1);

// Limpia todos los "trailing ones"
// 0011 0111 -> 0011 0000
n = n & (n+1)

// Enciende el último bit apagado
// 0011 0101 -> 0011 0111
n = n | (n+1)

// Extrae el último bit prendido
// 0011 0100 -> 0000 0100
n = n & ~n
```

```
// Número de bits encendidos
__builtin_popcount(x);
__builtin_popcountll(x);

// Posición del primer 1 desde la derecha (1-indexado)
__builtin_ffs(x);
__builtin_ffsll(x);

// 1 si número impar de bits encendidos
__builtin_parity(x);
__builtin_parityll(x);

// Nota:  clzll y ctzll no son seguras si x == 0

// Ceros a la izquierda después del primer bit encendido desde la
↪  derecha
//__builtin_clz(00000000 00000000 00000000 00010000) = 27
__builtin_clz(x);
__builtin_clzll(x);

// Ceros a la derecha despu+es del primer bit encendido desde la
↪  derecha
// __builtin_ctz(00000000 00000000 00000000 00010000) = 4
__builtin_ctz(x);
__builtin_ctzll(x);
```

Number Theory

Extended gcd

```
ll extended_gcd(ll a, ll b, ll& x, ll& y){
  if (b == 0){
    x = 1; y = 0;
    return a;
  }
  ll x1,y1;
  ll d = extended_gcd(b, a % b,x1,y1);
  x = y1;
  y = x1 - y1 * (a / b);
```

```
  return d;
}
```

Inverse mod

```
ll inverseMod(ll n, ll p){
    return binpow(n,p-2,p);
}
```

Is prime for large numbers

```
// Test de primalidad de Fermat (probabilistico)
bool isPrime(ll n, int iter = 5)
{
  if (n < 4) return n == 2 || n == 3;
  if (n % 2 == 0 || n % 3 == 0) return false;
  for (int i = 0; i < iter; i++)
  {
    ll a = 2 + rand() % (n - 3);
    if (modpow(a, n - 1, n) != 1) return false;
  }
  return true;
}
```

Modulo long long

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
constexpr ll MODL = 998244353ll;
struct mll
{
    ll x;
    mll() : x(0) {}
    mll(ll x_) : x((x_ % MODL) + (x_<0?MODL:0)) {}
    inline mll operator+(mll y) const { return mll(x + y.x); }
    inline mll operator*(mll y) const { return mll(x * y.x); }
    inline mll& operator+=(mll y) { x = (x + y.x) % MODL; return
    ↪ *this; }
    inline mll& operator*=(mll y) { x = (x * y.x) % MODL; return
    ↪ *this; }
    inline mll operator/(mll y) const { return mll(x *
    ↪ (y^(MODL-2)).x); }
    inline mll operator^(ll y) const
    {
        mll r = 1;
        mll a = mll(x);
        while (y > 0)
        {
            if (y & 1) r = r * a;
            a *= a;
            y >>= 1;
        }
        return r;
    }
};
ostream& operator<<(ostream& os, mll x) { os << x.x; return os; }
istream& operator>>(istream& is, mll& x) { ll y; is >> y; x = y;
↪ return is; }
```

NTT

```
// Transformada discreta de Fourier en Z_p
const int mod = 7340033;
const int root = 5; // (2^k)-ésima raíz de la unidad
const int root_1 = 4404020; // inversa de root (modulo mod)
const int root_pw = 1 << 20; // 2^k

// para p primo de la forma p=c2^k + 1, g^c es (2^k)-ésima raíz de la
↪ unidad,
// donde g es raíz primitiva de p, i.e. todo factor p_i de (p-1)
↪ cumple que
// g^((p-1)/p_i) mod p es congruente a 1.

// FFT + inverse FFT sobre Z_p
void fft(vector<int> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
```

```
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <<= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w %
                ↪  mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }
    }

    if (invert) {
        int n_1 = inverse(n, mod); // TODO: inverso modular de n
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}
```

Prime factorization

```
vector<ll> trial_division(ll n) {
  vector<ll> factorization;
  for (int d : {2, 3, 5}) {
    while (n % d == 0) {
      factorization.push_back(d);
      n /= d;
    }
  }
  static array<int, 8> increments = {4, 2, 4, 2, 4, 6, 2, 6};
  int i = 0;
  for (ll d = 7; d * d <= n; d += increments[i++]) {
    while (n % d == 0) {
      factorization.push_back(d);
      n /= d;
    }
    if (i == 8)
      i = 0;
  }
  if (n > 1)
    factorization.push_back(n);

  // Si factorization.size() == 1 entonces n es primo
  return factorization;
}
```

Data Structures

AVL

```
using ll = long long;
using i16 = short int;

// nodo avl
struct avl_n
{
  avl_n *l, *r; // izq/der
  i16 h; // altura
  ll sz; // tamaño subárbol
  int x; // valor

  // constructor
  avl_n(int x) : l(0), r(0), h(1), sz(1), x(x)
  { }

  // computa balance
  inline i16 balance() { return (r ? r->h : 0) - (l ? l->h : 0); }

  // encuentra el mínimo del subárbol
  avl_n *min()
  {
    avl_n *r = this;
```

```
    while (r->l)
      r = r->l;
    return r;
  }

  // encuentra el máximo del subárbol
  avl_n *max()
  {
    avl_n *r = this;
    while (r->r)
      r = r->r;
    return r;
  }
};

// arbol avl
struct avl_t
{
  // raíz
  avl_n *r;

  // constructor
  avl_t() : r(0)
  { }

  // auxiliar para computar el tamaño de un nodo posiblemente nulo
  inline int v_sz(avl_n *&v) { return v ? v->sz : 0; }
  // auxiliar para computar la altura de un nodo posiblemente nulo
  inline int v_h(avl_n *&v) { return v ? v->h : 0; }

  // actualiza altura y tamaño de un subárbol (vértice)
  inline void update(avl_n *&v)
  {
    if (!v)
      return;
    v->h = 1 + max(v_h(v->l), v_h(v->r));
    v->sz = 1 + v_sz(v->l) + v_sz(v->r);
  }

  // obtiene el tamaño del árbol
  inline int sz() { return v_sz(r); }

  // giro izq
  void lrot(avl_n *&v)
  {
    avl_n *y = v->r, *t = y->l;
    y->l = v, v->r = t;
    update(v), update(y);
    v = y;
  }

  // giro der
  void rrot(avl_n *&v)
  {
    avl_n *x = v->l, *t = x->r;
    x->r = v, v->l = t;
    update(v), update(x);
    v = x;
  }

  // actualiza el balance y realiza las rotaciones para mantenerlo
  void updateb(avl_n *&v)
  {
    if (!v)
      return;
    i16 b = v->balance();
    if (b > 1)
    {
      if (v->r->balance() < 0)
        rrot(v->r);
      lrot(v);
    }
    else if (b < -1)
    {
      if (v->l->balance() > 0)
        lrot(v->l);
      rrot(v);
    }
  }

  // inserta un nuevo valor al arbol
  void insert(avl_n *&v, int x)
  {
    if (v)
    {
      if (x < v->x)
        insert(v->l, x);
      else
```

```
      insert(v->r, x);
    update(v), updateb(v);
  }
  else
  {
    v = new avl_n(x);
  }
}

// busca si el valor se encuentra en el árbol
avl_n *search(int &x)
{
  avl_n *v = r;
  while (v)
  {
    if (x == v->x)
      break;
    v = (x < v->x ? v->l : v->r);
  }
  return v;
}

// borra el valor del árbol si existe
void erase(avl_n *&v, int &x)
{
  if (!v)
    return;
  if (x < v->x)
    erase(v->l, x);
  else if (x > v->x)
    erase(v->r, x);
  else
  {
    if (!v->l)
      v = v->r;
    else if (!v->r)
      v = v->l;
    else
    {
      v->x = v->r->min()->x;
      erase(v->r, v->x);
    }
  }
  update(v), updateb(v);
}

// inserta un valor al árbol
void insert(int x)
{
  insert(r, x);
}

// elimina un valor del árbol
void erase(int x)
{
  erase(r, x);
}

// encuentra el k-ésimo elemento
avl_n *kth(int i)
{
  avl_n *v = r;
  while (i != v_sz(v->l))
  {
    if (i < v_sz(v->l))
    {
      v = v->l;
    }
    else
    {
      i -= v_sz(v->l) + 1;
      v = v->r;
    }
  }
  return v;
}

// imprime la mediana (o su promedio si hay dos, o indica que no hay
↪  elementos
// suficientes para una mediana)
void print_median()
{
  if (sz() == 0)
  {
    cout << "Empty!\n";
  }
  else if (sz() % 2)
  {
```

```
      cout << kth(sz() / 2)->x << '\n';
    }
    else
    {
      avl_n *upmed = kth(sz() >> 1);
      avl_n *downmed = (upmed->l ? upmed->l->max() : kth((sz() >> 1) -
      ↪  1));
      cout << (downmed->x + ((upmed->x - downmed->x) >> 1)) << '\n';
    }
  }
};
```

Bitset

```
// Tamaño del bitset (usa hasta 32 si quieres convertir a int sin
↪  problemas)
const int N = 8;
int main() {
    //  Crear bitsets
    bitset<N> bs1(string("10101001"));   // Desde string binario
    int num = 42;
    bitset<N> bs2(num);                  // Desde entero (42 =
    ↪  00101010)

    cout << "bs1: " << bs1 << endl;
    cout << "bs2: " << bs2 << endl;

    //  Iterar sobre bits
    cout << "Bits activos (1) en bs1: ";
    for (int i = 0; i < N; ++i) {
        if (bs1[i]) cout << i << " ";
    }
    cout << "\n";

    //  Operaciones lógicas entre bitsets
    cout << "bs1 & bs2: " << (bs1 & bs2) << endl;
    cout << "bs1 | bs2: " << (bs1 | bs2) << endl;
    cout << "bs1 ^ bs2: " << (bs1 ^ bs2) << endl;

    //  Métodos importantes
    cout << "bs1.any():  " << bs1.any() << "  // ¿Algún bit en 1?\n";
    cout << "bs1.none(): " << bs1.none() << " // ¿Todos los bits en
    ↪  0?\n";
    cout << "bs1.all():  " << bs1.all() << "  // ¿Todos los bits en
    ↪  1?\n";
    cout << "bs1.count(): " << bs1.count() << " // ¿Cuántos bits en
    ↪  1?\n";

    //  Modificación de bits
    bs1.set(2);     // Pone a 1 el bit 2
    bs1.reset(0);   // Pone a 0 el bit 0
    bs1.flip(1);    // Invierte el bit 1

    cout << "bs1 modificado: " << bs1 << endl;

    bs1.set();      // Todos los bits a 1
    cout << "bs1.set(): " << bs1 << endl;

    bs1.reset();    // Todos los bits a 0
    cout << "bs1.reset(): " << bs1 << endl;

    bs1.flip();     // Invierte todos los bits
    cout << "bs1.flip(): " << bs1 << endl;

    //  Conversión a entero
    unsigned long val_ul = bs2.to_ulong();          // Hasta 32 bits
    unsigned long long val_ull = bs2.to_ullong();   // Hasta 64 bits
    int val_int = static_cast<int>(val_ul);         // Seguro si N <= 32

    cout << "bs2 como unsigned long: " << val_ul << endl;
    cout << "bs2 como int: " << val_int << endl;

    //  Convertir int a bitset nuevamente
    bitset<N> fromInt(val_int);
    cout << "Convertido desde int: " << fromInt << endl;

    return 0;
}
```

Deque

```
int main() {
    deque<int> dq;
```

```
    // Insertar elementos al final de la deque (push_back)
    dq.push_back(10);
    dq.push_back(20);
    dq.push_back(30);
    dq.push_back(40);

    // Insertar elementos al frente de la deque (push_front)
    dq.push_front(5);
    dq.push_front(2);

    // Mostrar los elementos de la deque
    cout << "Elementos en la deque:" << endl;
    for (int el : dq) {
        cout << el << " ";
    }
    cout << endl;

    // Eliminar el elemento al frente (pop_front)
    dq.pop_front();

    // Eliminar el elemento al final (pop_back)
    dq.pop_back();

    cout << "Elementos después de eliminar el frente y el final:" <<
    ↪  endl;
    for (int el : dq) {
        cout << el << " ";
    }
    cout << endl;

    // Verificar si la deque está vacía
    if (dq.empty()) {
        cout << "La deque está vacía." << endl;
    } else {
        cout << "La deque no está vacía." << endl;
    }

    // Mostrar el primer y último elemento
    cout << "El primer elemento es: " << dq.front() << endl;
    cout << "El último elemento es: " << dq.back() << endl;
    return 0;
}
```

DSU

```
typedef vector<int> vi;
/**
 * Estructura DSU (Disjoint Set Union) o Union-Find.
 * Permite mantener conjuntos disjuntos con operaciones eficientes.
 */
struct DSU
{
  vi parent;   // parent[i]: padre del nodo i
  vi rank;     // rank[i]: estimación de la profundidad del árbol del
  ↪  conjunto que contiene a i
  vi setSize;  // setSize[i]: tamaño del conjunto cuya raíz es i
  int numSets; // Número actual de conjuntos disjuntos

  // Constructor: Inicializa DSU con N elementos (1 a N)
  DSU(int N)
  {
    N++; // Para trabajar con índices desde 1
    parent.resize(N);
    rank.assign(N, 0);
    setSize.assign(N, 1);
    for (int i = 1; i < N; ++i)
      parent[i] = i;
    numSets = N - 1;
  }

  // Devuelve la raíz del conjunto que contiene a 'i' (con compresión
  ↪  de caminos)
  int find(int i)
  {
    if (parent[i] != i)
      parent[i] = find(parent[i]);
    return parent[i];
  }

  // Retorna true si 'i' y 'j' están en el mismo conjunto
  bool isSameSet(int i, int j)
  {
    return find(i) == find(j);
  }

  // Retorna el número de conjuntos disjuntos actuales
```

```
  int countSets()
  { return numSets; }

  // Retorna el tamaño del conjunto que contiene al elemento 'i'
  int size(int i)
  { return setSize[find(i)]; }

  // Une los conjuntos que contienen a 'i' y 'j'
  void unify(int i, int j)
  {
    if (isSameSet(i, j))
      return;
    int x = find(i), y = find(j);

    // Unión por rango para mantener árboles bajos
    if (rank[x] > rank[y])
      swap(x, y);

    parent[x] = y;
    if (rank[x] == rank[y])
      rank[y]++;

    setSize[y] += setSize[x];
    --numSets;
  }
};

int main()
{
  // Ejemplo de uso
  DSU dsu(5); // Elementos del 1 al 5

  dsu.unify(1, 2);
  dsu.unify(3, 4);

  cout << dsu.isSameSet(1, 2) << endl; //  1 (true)
  cout << dsu.isSameSet(1, 3) << endl; //  0 (false)
  cout << dsu.size(1) << endl;         //  2
  cout << dsu.countSets() << endl;     //  3

  dsu.unify(2, 3);
  cout << dsu.countSets() << endl; //  2

  // Ejemplo componentes conexas
  int n = 6; // nodos del 1 al 6
  vector<pair<int, int>> edges = {
      {1, 2}, {2, 3}, {4, 5}};

  DSU dsu(n);

  // Unir nodos conectados
  for (auto &[u, v] : edges)
    dsu.unify(u, v);

  // Mostrar cuántas componentes conexas hay
  cout << "Componentes conexas: " << dsu.countSets() << endl; //
  ↪  Debería imprimir 3

  // Mostrar tamaño de la componente de cada nodo
  for (int i = 1; i <= n; ++i)
  {
    cout << "Nodo " << i << " pertenece a componente de tamaño: " <<
    ↪  dsu.size(i) << endl;
  }
}
```

List

```
#include <iostream>
#include <list>
#include <iterator> // Necesario para next() y prev()
using namespace std;

int main() {
    list<int> lst;
    lst.push_back(10);
    lst.push_back(20);
    lst.push_back(30);
    lst.push_back(40);
    lst.push_back(50);

    cout << "Elementos en la lista (usando iteradores):" << endl;
    for (auto it = lst.begin(); it != lst.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;
```

```
    auto it = lst.begin();
    cout << "El primer elemento es: " << *it << endl;
    it = next(it, 2); // Avanzar el iterador 2 posiciones
    cout << "El tercer elemento (después de avanzar 2 pasos) es: " <<
    ↪  *it << endl;

    // Uso de `prev()` para retroceder el iterador
    it = prev(it, 1); // Retroceder 1 posición
    cout << "El segundo elemento (después de retroceder 1 paso) es: "
    ↪  << *it << endl;

    // Inserción de un elemento antes de un iterador específico
    it = next(lst.begin(), 2); // Moverse al tercer elemento
    lst.insert(it, 35); // Insertar 35 antes del tercer elemento

    // Mostrar los elementos después de la inserción
    cout << "\nElementos después de insertar 35 antes del tercer
    ↪  elemento:" << endl;
    for (auto it = lst.begin(); it != lst.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // Eliminar un elemento usando un iterador
    it = next(lst.begin(), 3); // Moverse al cuarto elemento
    lst.erase(it); // Eliminar el cuarto elemento (40)

    // Acceder al primer y último elemento usando `front()` y `back()`
    cout << "\nEl primer elemento es: " << lst.front() << endl;
    cout << "El último elemento es: " << lst.back() << endl;
    return 0;
}
```

MinMax Heap

```
// Ojo. las operaciones de lectura/eliminacion asumen que si hay cosas
#include <bits/stdc++.h>
using namespace std;
struct MMH{
    vector<int> h;
    bool minLv(int i){int d=0;for(;i;i=(i-1)/2)d++;return d%2==0;}
    void push(int x){
        h.push_back(x);int i=h.size()-1,p=(i-1)/2;
        if((minLv(i)&&h[i]>h[p])||(!minLv(i)&&h[i]<h[p]))
            swap(h[i],h[p]),minLv(i)?upMax(p):upMin(p);
        else minLv(i)?upMin(i):upMax(i);
    }
    int getMin(){return h[0];}
    int getMax(){return
    ↪  h.size()<2?h[0]:(h.size()==2?h[1]:max(h[1],h[2]));}
    void popMin(){h[0]=h.back();h.pop_back();down(0);}
    void popMax(){int m=h.size()==2||h[1]>h[2]?1:2;
        h[m]=h.back();h.pop_back();down(m);}
    void upMin(int i){while(i>=3){int g=((i-1)/2-1)/2;
        if(h[i]<h[g])swap(h[i],h[g]),i=g;else break;}}
    void upMax(int i){while(i>=3){int g=((i-1)/2-1)/2;
        if(h[i]>h[g])swap(h[i],h[g]),i=g;else break;}}
    void down(int i){minLv(i)?downMin(i):downMax(i);}
    void downMin(int i){for(;;){int m=-1;
        for(int d=1;d<=4;d++){int c=2*i+d;
            if(c<h.size()&&(m==-1||h[c]<h[m]))m=c;}
        if(m==-1)break;
        if(m>=4*i+3){if(h[m]<h[i]){swap(h[m],h[i]);
            int p=(m-1)/2;if(h[m]>h[p])swap(h[m],h[p]);downMin(m);}}
        else{if(h[m]<h[i])swap(h[m],h[i]);else break;}}}
    void downMax(int i){for(;;){int m=-1;
        for(int d=1;d<=4;d++){int c=2*i+d;
            if(c<h.size()&&(m==-1||h[c]>h[m]))m=c;}
        if(m==-1)break;
        if(m>=4*i+3){if(h[m]>h[i]){swap(h[m],h[i]);
            int p=(m-1)/2;if(h[m]<h[p])swap(h[m],h[p]);downMax(m);}}
        else{if(h[m]>h[i])swap(h[m],h[i]);else break;}}}
};
```

Monotonic MeteSaca

```
struct mono_stack
{
    vector<pair<int,int>> st;

    void push(int v, int idx)
    {
```

```
        // cambiar <= a < si quieres ser estricto
        while (!st.empty() && st.back().first <= v)
            st.pop_back();
        st.emplace_back(v, idx);
    }

    void pop()
    {
        if (!st.empty())
            st.pop_back();
    }

    int top() const // verificar primero empty()!!!
    {
        return st.back().first;
    }

    bool empty() const { return st.empty(); }
    size_t size() const { return st.size(); }
};

struct mono_queue
{
    deque<pair<int,int>> dq; // {valor, idx}

    void push(int v, int idx)
    {
        // cambiar <= a < si quieres ser estricto
        while (!dq.empty() && dq.back().first <= v)
            dq.pop_back();
        dq.push_back({v,idx});
    }

    void pop(int idx_out)
    {
        // cambiar <= a < si quieres ser estricto
        if (!dq.empty() && dq.front().second <= idx_out)
            dq.pop_front();
    }

    int get() const // verificar primero empty()!!!
    {
        return dq.front().first;
    }
};

// Comparadores estándar: less<T>, less_equal<T>, greater<T>,
↪  greater_equal<T>
template<typename T, typename Compare = less<T>>
struct mono_deque
{
    deque<T> dq;
    Compare comp;

    void push_back_mono(const T& val)
    {
        while (!dq.empty() && comp(dq.back(), val))
            dq.pop_back();
        dq.push_back(val);
    }

    void push_front_mono(const T& val)
    {
        while (!dq.empty() && comp(val, dq.front()))
            dq.pop_front();
        dq.push_front(val);
    }

    void pop_back() {dq.pop_back();} // verificar empty()!!!
    void pop_front() {dq.pop_front();} // verificar empty()!!!
    const T& front() const {return dq.front();} // verificar empty()!!!
    const T& back() const {return dq.back();} // verificar empty()!!!
    bool empty() const {return dq.empty();}
    size_t size() const {return dq.size();}
};
```

Ordered Set

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

// Se puede usar como un set, tiene todas sus funcionalidades pero
↪  agrega dos funciones extra
```

```cpp
template <class T>
using oset = tree<T, null_type, less<T>,
                  rb_tree_tag, tree_order_statistics_node_update>;
// find_by_order, order_of_key
// También en vez de less<T> se puede poner greater<T> mayor estricto
↪  (>)

int main()
{
  oset<int> A; // declaración

  // Insertar elementos
  A.insert(1);
  A.insert(10);
  A.insert(2);
  A.insert(7);
  A.insert(2); // Solo contiene valores unicos

  // A contains = 1 2 7 10
  cout << "A = ";
  for (auto i : A)
    cout << i << " ";
  cout << endl;

  // Encontrar el kth elemento
  cout << "0th element: " << *A.find_by_order(0) << endl; //  1
  cout << "1st element: " << *A.find_by_order(1) << endl; //  2
  cout << "2nd element: " << *A.find_by_order(2) << endl; //  7
  cout << "3rd element: " << *A.find_by_order(3) << endl; //  10
  auto invalido = A.find_by_order(10);
  if (invalido == A.end())
    cout << "No existe ese elemento" << endl;
  cout << endl;

  // Encontrar el número de elementos menores que x
  cout << "No. of elems smaller than 6: " << A.order_of_key(6) <<
  ↪  endl;   //  2
  cout << "No. of elems smaller than 11: " << A.order_of_key(11) <<
  ↪  endl; //  4
  cout << "No. of elems smaller than 1: " << A.order_of_key(1) <<
  ↪  endl;   //  0
  cout << endl;

  // lower bound -> Primer elemento >= que x
  cout << "Lower Bound of 6: " << *A.lower_bound(6) << endl;
  cout << "Lower Bound of 2: " << *A.lower_bound(2) << endl;
  cout << endl;

  // Upper bound -> Primer elemento > que x
  cout << "Upper Bound of 6: " << *A.upper_bound(6) << endl;
  cout << "Upper Bound of 1: " << *A.upper_bound(1) << endl;
  cout << endl;

  // Eliminar elementos
  A.erase(1);
  A.erase(11); // Eliminar un elemento que no está no hace nada

  // A contains = 2 7 10
  cout << "A = ";
  for (auto i : A)
    cout << i << " ";
  cout << endl;
}
```

Priority Queue

```cpp
struct Comparador
{
  bool operator()(const pair<int, string> &a, const pair<int, string>
  ↪  &b)
  {
    return a.first > b.first; // Ordenar por el primer valor del par
    ↪  (min-heap por valor de entero)
  }
};

int main()
{
  // Crear una priority_queue con el comportamiento por defecto
  ↪  (max-heap)
  priority_queue<int> pqMax;

  // Insertar elementos en el max-heap
  pqMax.push(5);
  pqMax.push(2);
  pqMax.push(8);
  pqMax.push(1);
```

```
pqMax.push(3);

cout << "Elementos en el max-heap (prioridad alta a baja):" << endl;

// Extraer elementos (se extrae el de mayor valor primero)
// 8 5 3 2 1
while (!pqMax.empty())
{
  cout << pqMax.top() << " "; // Muestra el elemento de mayor
  ↪  prioridad (en max-heap, el más grande)
  pqMax.pop();                // Elimina el elemento de mayor
  ↪  prioridad
}
cout << endl;

// Crear una priority_queue con el comportamiento de min-heap
priority_queue<int, vector<int>, greater<int>> pqMin;

// Insertar elementos en la min-heap
pqMin.push(5);
pqMin.push(2);
pqMin.push(8);
pqMin.push(1);
pqMin.push(3);

cout << "Elementos en el min-heap (prioridad baja a alta):" << endl;

// Extraer elementos (se extrae el de menor valor primero)
// 1 2 3 5 8
while (!pqMin.empty())
{
  cout << pqMin.top() << " "; // Muestra el elemento de mayor
  ↪  prioridad (en max-heap, el más grande)
  pqMin.pop();                // Elimina el elemento de mayor
  ↪  prioridad
}
cout << endl;

// Priority Queue con comparador persoanlizado
priority_queue<pair<int, string>, vector<pair<int, string>>,
↪  Comparador> pq;

pq.push({5, "cinco"});
pq.push({2, "dos"});
pq.push({8, "ocho"});
pq.push({1, "uno"});
pq.push({3, "tres"});

while (!pq.empty())
{
  cout << pq.top().first << " - " << pq.top().second << endl;
  pq.pop();
}
// 1 - uno
// 2 - dos ...
return 0;
}
```

LazySegmentTree

Assing On A Range And Look On Position

```
// Lazy segment tree para queries de asignamiento en rango
// y buscar en una posicion
template <typename T>
struct LazySegmentTree
{
  int n;
  vector<T> st, lazy;
  vector<bool> marked;

  void push(int v)
  {
    if (marked[v])
    {
      lazy[2 * v] = lazy[2 * v + 1] = lazy[v];
      marked[2 * v] = marked[2 * v + 1] = true;
      lazy[v] = 0;
      marked[v] = false;
    }
  }

  void build(vector<T> &nums, int v, int tl, int tr)
  {
    if (tl == tr)
      st[v] = nums[tl];
    else
```

```
    {
      int mid = (tl + tr) / 2;
      build(nums, 2 * v, tl, mid);
      build(nums, 2 * v + 1, mid + 1, tr);
    }
  }

  LazySegmentTree(vector<T> &nums)
  {
    n = nums.size();
    st.resize(4 * n);
    lazy.resize(4 * n);
    marked.resize(4 * n);
    build(nums, 1, 0, n - 1);
  }

  T query(int v, int tl, int tr, int pos)
  {
    if (tl == tr)
      return marked[v] ? lazy[v] : st[v];
    push(v);
    int mid = (tl + tr) / 2;
    if (pos <= mid)
      return query(2 * v, tl, mid, pos);
    return query(2 * v + 1, mid + 1, tr, pos);
  }

  void update(int v, int tl, int tr, int l, int r, T val)
  {
    if (tl > r || tr < l)
      return;
    if (tl >= l && tr <= r)
      lazy[v] = val, marked[v] = true;
    else
    {
      push(v);
      int mid = (tl + tr) / 2;
      update(2 * v, tl, mid, l, r, val);
      update(2 * v + 1, mid + 1, tr, l, r, val);
    }
  }
};

int main()
{
  int n, q; cin >> n >> q;
  vll nums(n);
  LazySegmentTree<ll> st(nums);
  while (q--)
  {
    int op;
    cin >> op;
    if (op == 1)
    {
      int l, r; ll v;
      cin >> l >> r >> v;
      l--; r--; // Solo restamos si las posiciones son 1-indexed
      st.update(1, 0, n-1, l, r, v);
    }
    else
    {
      int pos; cin >> pos;
      pos--; // Solo restamos si las posiciones son 1-indexed
      cout << st.query(1, 0, n-1, pos) << "\n";
    }
  }
  return 0;
}
```

Sum And Update On Range

```
// Lazy Segment Tree para suma de un valor x en rangos
// y obtener la suma de un rango [l, r]
template <typename T>
struct LazySegmentTree
{
  T neuter;
  vector<ll> st;
  vector<ll> lazy;

  LazySegmentTree(vector<T> &nums)
  {
    neuter = 0;
    int n = nums.size();
    st.assign(4 * n, neuter);
    lazy.assign(4 * n, 0);
    build(1, 0, n - 1, nums);
  }
```

```
inline T merge(T a, T b) { return a + b; }

void push(int v, int tl, int tr)
{
  lazy[v * 2] += lazy[v];
  lazy[v * 2 + 1] += lazy[v];
  int tm = tl + (tr - tl) / 2;
  st[v * 2] += lazy[v] * ((tm - tl) + 1);
  st[v * 2 + 1] += lazy[v] * (tr - tm);
  lazy[v] = 0;
}
void build(int v, int tl, int tr, vector<T> &nums)
{
  if (tl == tr)
  {
    st[v] = nums[tl];
    return;
  }
  int tm = tl + (tr - tl) / 2;
  build(v * 2, tl, tm, nums);
  build(v * 2 + 1, tm + 1, tr, nums);
  st[v] = merge(st[v * 2], st[v * 2 + 1]);
}

void update(int v, int tl, int tr, int l, int r, T val)
{
  if (tl > r || tr < l) return;

  if (tl >= l && tr <= r)
  {
    st[v] += val * ((tr - tl) + 1);
    lazy[v] += val;
    return;
  }
  push(v, tl, tr);
  int tm = tl + (tr - tl) / 2;
  update(v * 2, tl, tm, l, r, val);
  update(v * 2 + 1, tm + 1, tr, l, r, val);
  st[v] = merge(st[v * 2], st[v * 2 + 1]);
}

T query(int v, int tl, int tr, int l, int r)
{
    if (tl > r || tr < l) return neuter;


    if (tl >= l && tr <= r) return st[v];

    push(v, tl, tr);
    int tm = tl + (tr - tl) / 2;
    return merge(query(v * 2, tl, tm, l, r),
           query(v * 2 + 1, tm + 1, tr, l, r));
  }
};
```

Sum On A Range And Minimum

```
// Lazy segment tree para querys de sumar en un rango
// y encontrar el minimo de un rango [l, r]
template <typename T>
struct LazySegmentTree
{
  int n;
  int inf = INT_MAX;
  vector<T> st, lazy;

  void push(int v)
  {
    st[2 * v] += lazy[v];
    lazy[2 * v] += lazy[v];
    st[2 * v + 1] += lazy[v];
    lazy[2 * v + 1] += lazy[v];
    lazy[v] = 0;
  }

  void build(vector<T> &nums, int v, int tl, int tr)
  {
    if (tl == tr)
      st[v] = nums[tl];
    else
    {
      int mid = (tl + tr) / 2;
      build(nums, 2 * v, tl, mid);
      build(nums, 2 * v + 1, mid + 1, tr);
    }
  }
```

```
  LazySegmentTree(vector<T> &nums)
  {
    n = nums.size();
    st.resize(4 * n);
    lazy.resize(4 * n);
    build(nums, 1, 0, n - 1);
  }

  T query(int v, int tl, int tr, int l, int r)
  {
    if (tl > r || tr < l)
      return inf;
    if (tl >= l && tr <= r)
      return st[v];
    push(v);
    int mid = (tl + tr) / 2;
    return min(query(2 * v, tl, mid, l, r),
               query(2 * v + 1, mid + 1, tr, l, r));
  }

  void update(int v, int tl, int tr, int l, int r, T val)
  {
    if (tl > r || tr < l)
      return;
    if (tl >= l && tr <= r)
      lazy[v] += val, st[v] += val;
    else
    {
      push(v);
      int mid = (tl + tr) / 2;
      update(2 * v, tl, mid, l, r, val);
      update(2 * v + 1, mid + 1, tr, l, r, val);
    }
    st[v] = min(st[2 * v], st[2 * v + 1]);
  }
};
```

MergeSortTree

MergeSortTree

```
template <typename T>
struct MergeSortTree
{
  vector<vector<T>> st;
  MergeSortTree(vector<T> &nums)
  {
    st.resize(nums.size() * 4);
    build(1, 0, nums.size() - 1, nums);
  }

  void build(int v, int tl, int tr, vector<T> &nums)
  {
    if (tl == tr)
    {
      st[v] = {nums[tl]};
      return;
    }

    int tm = tl + (tr - tl) / 2;
    build(v * 2, tl, tm, nums);
    build(v * 2 + 1, tm + 1, tr, nums);

    merge(all(st[v * 2]), all(st[v * 2 + 1]), back_inserter(st[v]));
  }

  // Número de elementos iguales a val
  T query(int v, int tl, int tr, int l, int r, T val)
  {
    if (tl > r || tr < l)
    {
      return 0;
    }

    if (tl >= l && tr <= r)
    {
      auto it = lower_bound(all(st[v]), val);
      auto jt = upper_bound(all(st[v]), val);

      if (it == st[v].end())
      {
        return 0;
      }

      int j = jt - st[v].begin();
      int i = it - st[v].begin();
      return j - i;
```

```
    }

    int tm = tl + (tr - tl) / 2;

    return query(v * 2, tl, tm, l, r, val) + query(v * 2 + 1, tm + 1,
    ↪  tr, l, r, val);
  }
};
```

SegmentTree

2D Segment Tree

```
// Primero se aplica cualquier cosa sobre X y después sobre Y. Esto
↪  aplica a
// todas las funciones siguientes, por lo que se debe llamar su
↪  versión en X.

// La entrada es una matriz a[][], y el árbol se almacena sobre t[][]

// CONSTRUCCIÓN

// Esto es interno, se debe llamar build_x el cual llamará esta función
// Build the Y-axis segment tree for a fixed X-node vx
// vx: index in X-axis segment tree
// lx, rx: X-range covered by vx
// vy: index in Y-axis segment tree for this vx
// ly, ry: Y-range covered by vy
void build_y(int vx, int lx, int rx, int vy, int ly, int ry) {
    if (ly == ry)
        if (lx == rx)
            t[vx][vy] = a[lx][ly];
        else
            t[vx][vy] = t[vx*2][vy] + t[vx*2+1][vy];
    else {
        int my = (ly + ry) / 2;
        build_y (vx, lx, rx, vy*2, ly, my);
        build_y (vx, lx, rx, vy*2+1, my+1, ry);
        t[vx][vy] = t[vx][vy*2] + t[vx][vy*2+1];
    }
}

// Sobre el segment-tree en X se construyen segment trees en Y
// Build the X-axis segment tree
// vx: index in X-axis segment tree
// lx, rx: X-range covered by vx
void build_x(int vx, int lx, int rx) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        build_x (vx*2, lx, mx);
        build_x (vx*2+1, mx+1, rx);
    }
    build_y (vx, lx, rx, 1, 0, m-1);
}

// EJEMPLO DE QUERY

// Esto es interno, realmente se debería llamar sum_x, pues se procesa
↪  primero
// un strip en X y después los strips en Y.
// Query the Y-axis segment tree for a fixed X-node vx
// vx: fixed X-node index
// vy: current Y-node index
// tly, try_: total Y-range covered by vy
// ly, ry: query Y-range
int sum_y(int vx, int vy, int tly, int try_, int ly, int ry) {
    if (ly > ry)
        return 0;
    if (ly == tly && try_ == ry)
        return t[vx][vy];
    int tmy = (tly + try_) / 2;
    return sum_y (vx, vy*2, tly, tmy, ly, min(ry,tmy))
        + sum_y (vx, vy*2+1, tmy+1, try_, max(ly,tmy+1), ry);
}

// O(logn logm) pues se hacen todos los strips en Y sobre el strip en X
// Query the X-axis segment tree
// vx: current X-node index
// tlx, trx: total X-range covered by vx
// lx, rx: query X-range
// ly, ry: query Y-range
int sum_x(int vx, int tlx, int trx, int lx, int rx, int ly, int ry) {
    if (lx > rx)
        return 0;
    if (lx == tlx && trx == rx)
        return sum_y (vx, 1, 0, m-1, ly, ry);
    int tmx = (tlx + trx) / 2;
    return sum_x (vx*2, tlx, tmx, lx, min(rx,tmx), ly, ry)
```

```
        + sum_x (vx*2+1, tmx+1, trx, max(lx,tmx+1), rx, ly, ry);
}

// EJEMPLO DE UPDATE

// De nuevo, esto es interno, se debería llamar update_x
// Update the Y-axis segment tree for a fixed X-node vx
// vx: fixed X-node index
// lx, rx: X-range covered by vx
// vy: current Y-node index
// ly, ry: Y-range covered by vy
// x, y: coordinates to update
// new_val: new value to set at (x, y)
void update_y (int vx, int lx, int rx, int vy, int ly, int ry, int x,
↪  int y, int new_val) {
    if (ly == ry) {
        if (lx == rx)
            t[vx][vy] = new_val;
        else
            t[vx][vy] = t[vx*2][vy] + t[vx*2+1][vy];
    }
    else {
        int my = (ly + ry) / 2;
        if (y <= my)
            update_y (vx, lx, rx, vy*2, ly, my, x, y, new_val);
        else
            update_y (vx, lx, rx, vy*2+1, my+1, ry, x, y, new_val);
        t[vx][vy] = t[vx][vy*2] + t[vx][vy*2+1];
    }
}

// O(logn logm + k)
// Update the X-axis segment tree
// vx: current X-node index
// lx, rx: X-range covered by vx
// x, y: coordinates to update
// new_val: new value to set at (x, y)
void update_x (int vx, int lx, int rx, int x, int y, int new_val) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        if (x <= mx)
            update_x (vx*2, lx, mx, x, y, new_val);
        else
            update_x (vx*2+1, mx+1, rx, x, y, new_val);
    }
    update_y (vx, lx, rx, 1, 0, m-1, x, y, new_val);
}
```

Segment Tree

```
template <typename T>
struct SegmentTree
{
  // usa, v = 1, tl = 0, tr = n-1 para 0-indexed
  // usa  v = 1, tl = 1, tr = n para 1-indexed pero
  // asegurate de acceder a nums[tl] correctamente
  T neuter;
  vector<T> st;
  inline T merge(T a, T b) { return a + b; }

  SegmentTree(vector<T> &nums)
  {
    neuter = 0;
    st.assign(4 * nums.size(), neuter);
    build(1, 0, nums.size() - 1, nums);
  }

  void build(int v, int tl, int tr, vector<T> &nums)
  {
    if (tl == tr)
    {
      st[v] = nums[tl];
      return;
    }
    int tm = tl + (tr - tl) / 2;
    build(v * 2, tl, tm, nums);
    build(v * 2 + 1, tm + 1, tr, nums);
    st[v] = merge(st[2 * v], st[2 * v + 1]);
  }
```

```
  void update(int v, int tl, int tr, int pos, T val)
  {
    if (tl == tr)
    {
      st[v] = val;
      return;
    }
    int tm = tl + (tr - tl) / 2;
    if (pos <= tm) update(v * 2, tl, tm, pos, val);
    else update(v * 2 + 1, tm + 1, tr, pos, val);
    st[v] = merge(st[v * 2], st[v * 2 + 1]);
  }

  T query(int v, int tl, int tr, int l, int r)
  {
    if (tl >= l && tr <= r) return st[v];
    if (tl > r || tr < l) return neuter;
    int tm = tl + (tr - tl) / 2;
    return merge(query(v * 2, tl, tm, l, r),
    query(v * 2 + 1, tm + 1, tr, l, r));
  }
};
```

Graphs

Flows

Closure Problem

```
/**
 *
 * Problema de cerradura/clausura máxima (o mínima):
 *
 * Una cerradura C de una digráfica D es un conjunto de V(D) tal que
 ↪  no hay
 * flecha que salga de C.
 *
 * El problema de clausura máxima (o análogamente mínima) se trata de
 ↪  encontrar
 * C de peso máximo (o análogamente mínimo), con pesos sobre los
 ↪  vértices.
 *
 * Para pasar de un problema de clausura mínima a clausura máxima, la
 ↪  clausura
 * mínima es equivalente a la clausura máxima de la digráfica conversa
 ↪  (i.e. con
 * las flechas al revés).
 *
 * Para resolver el problema y encontrar el peso de la cerradura
 ↪  máxima se construye
 * la red de flujo con las mismas flechas que D y peso infinito, con
 ↪  los vértices
 * nuevos S y T, que todo vértice v con peso positivo tiene una flecha
 ↪  de S a v
 * con el peso de v, y todo vértice v con peso negativo tiene una
 ↪  flecha de v a
 * T con el valor absoluto del peso de v.
 *
 * El peso de la cerradura máxima es la suma de todos los pesos
 ↪  positivos menos
 * el flujo máximo de la red. Esto corresponde con el corte mínimo
 ↪  necesario
 * del valor máximo, i.e. que es lo menos que podemos "sacrificar" del
 ↪  peso
 * máximo posible.
 *
 * Los vértices en la cerradura máxima son aquellos que están del lado
 ↪  de S en
 * el corte mínimo.
 *
 */
using dig_adj = vector<vector<ll>>;
constexpr ll inf = 1e17;

ll peso_cerradura_max(ll n, dig_adj g, vector<ll> costs)
{
    // Le asignamos a S y T los índices n y n+1
    ll s = n, t = n + 1;
    Dinic flujo(n + 2, s, t);
    // Agrega las aristas en la gráfica
    for (ll v = 0; v < n; v++)
    for (ll u : g[v])
    {
        flujo.add(v, u, inf);
    }
    // Agrega los costos
```

```
    for (ll v = 0; v < n; v++)
    {
        ll c = costs[v];
        if (c > 0)
        {
            flujo.add(s, v, c);
        }
        else if (c < 0)
        {
            flujo.add(v, t, -c);
        }
    }

    // Suma de costos
    ll suma = 0;
    for (ll c : costs) if (c > 0) suma += c;

    return suma - flujo.maxFlow();
}
```

Flow

```
#include <bits/stdc++.h>

using namespace std;
typedef long long ll;
typedef vector<int> vi;
typedef pair<int,int> pii;
typedef vector<pii> vii;

#define all(v) v.begin(),v.end()
#define pb(x) push_back(x)

const ll inf = 1e17;

struct flowEdge {
    int u,v;
    ll cap,flow = 0;
    flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
};

struct Dinic{
    vector<flowEdge> edges;
    vector<vi> adj;
    int n,s,t;
    int id = 0;
    vi level, next;

    queue<int> q;
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        next.resize(n);
        fill(all(level),-1);
        level[s] = 0;
        q.push(s);
    }

    void add(int u, int v, ll cap){
        edges.emplace_back(u,v,cap);
        edges.emplace_back(v,u,0);
        adj[u].pb(id++);
        adj[v].pb(id++);
    }

    bool bfs(){
        while (!q.empty()){
            int curr = q.front();
            q.pop();
            for (auto e : adj[curr]){
                if (edges[e].cap - edges[e].flow  < 1) continue;
                if (level[edges[e].v] != -1) continue;
                level[edges[e].v] = level[edges[e].u] + 1;
                q.push(edges[e].v);
            }
        }
        return level[t] != -1;
    }

    ll dfs(int u, ll flow){
        if (flow == 0) return 0;
        if (u == t) return flow;
        for (auto& cid = next[u]; cid < adj[u].size(); cid++){
            int e = adj[u][cid];
            int v = edges[e].v;
```

```
            if (level[edges[e].u] + 1 != level[v] || edges[e].cap -
            ↪  edges[e].flow < 1) continue;
            ll f = dfs(v,min(flow,edges[e].cap - edges[e].flow));
            if (f == 0) continue;
            edges[e].flow += f;
            edges[e ^ 1].flow -= f;
            return f;
        }
        return 0;
    }

    ll maxFlow(){
        ll flow = 0;
        while (bfs()){
            fill(all(next),0);
            for (ll f = dfs(s,inf); f != 0ll; f = dfs(s,inf)) flow +=
            ↪  f;
            fill(all(level),-1);
            level[s] = 0;
            q.push(s);
        }
        return flow;
    }

};

int main(){
    int n,m; cin >> n >> m;
    Dinic g(n,0,n-1);
    for(int i = 0; i < m ; i++){
        int a,b,c; cin >> a >> b >> c;
        g.add(a, b, c);
    }
    cout << g.maxFlow() << '\n';
}
```

Min Cost Max Flow (Dinic)

```
// O(Fmlogn) (avg case ~O(sqrt(n)mlogn))
/**
 * Author: Stanford
 * Date: Unknown
 * Source: Stanford Notebook
 * Description: Min-cost max-flow.
 *  If costs can be negative, call setpi before maxflow, but note that
 ↪  negative
 *      cost cycles are not supported.
 *  To obtain the actual flow, look at positive values only.
 * Status: Tested on kattis:mincostmaxflow, stress-tested against
 ↪  another
 *      implementation
 * Time: $O(F E \log(V))$ where F is max flow. $O(VE)$ for setpi.
 */

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

// #include <bits/extc++.h> /// include-line, keep-include

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
```

```
    q.push({ 0, s });

    while (!q.empty()) {
        s = q.top().second; q.pop();
        seen[s] = 1; di = dist[s] + pi[s];
        for (edge& e : ed[s]) if (!seen[e.to]) {
            ll val = di - pi[e.to] + e.cost;
            if (e.cap - e.flow > 0 && val < dist[e.to]) {
                dist[e.to] = val;
                par[e.to] = &e;
                if (its[e.to] == q.end())
                    its[e.to] = q.push({ -dist[e.to], e.to });
                else
                    q.modify(its[e.to], { -dist[e.to], e.to });
            }
        }
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
          for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
        assert(it >= 0); // negative cost cycle
    }
};
```

Min Cost Max Flow (Edmond Karps)

```
// O(Fnm)
struct Edge
{
    int from, to, capacity, cost;
};

vector<vector<int>> adj, cost, capacity;

const int INF = 1e9;

void shortest_paths(int n, int v0, vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(v0);
    p.assign(n, -1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]) {
                d[v] = d[u] + cost[u][v];
                p[v] = u;
                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
}
```

```
int min_cost_flow(int N, vector<Edge> edges, int K, int s, int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N, 0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.capacity;
    }

    int flow = 0;
    int cost = 0;
    vector<int> d, p;
    while (flow < K) {
        shortest_paths(N, s, d, p);
        if (d[t] == INF)
            break;

        // find max flow on that path
        int f = K - flow;
        int cur = t;
        while (cur != s) {
            f = min(f, capacity[p[cur]][cur]);
            cur = p[cur];
        }

        // apply flow
        flow += f;
        cost += f * d[t];
        cur = t;
        while (cur != s) {
            capacity[p[cur]][cur] -= f;
            capacity[cur][p[cur]] += f;
            cur = p[cur];
        }
    }

    if (flow < K)
        return -1;
    else
        return cost;
}
```

Min Cut

```
/**
 * Author: Simon Lindholm
 * Date: 2021-01-09
 * License: CC0
 * Source: https://en.wikipedia.org/wiki/Stoer%E2%80%93Wagner_algorithm
 * Description: Find a global minimum cut in an undirected graph, as
 *              represented by an adjacency matrix.
 * Time: O(V^3)
 * Status: Stress-tested together with GomoryHu
 */
#pragma once

pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i,0,n) co[i] = {i};
    rep(ph,1,n) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i,0,n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i,0,n) mat[s][i] += mat[t][i];
        rep(i,0,n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

Push Relabel Flow

```
/**
 * Author: Simon Lindholm
```

```
 * Date: 2015-02-24
 * License: CC0
 * Source: Wikipedia, tinyKACTL
 * Description: Push-relabel using the highest label selection rule
 ↪  and the gap
 *     heuristic. Quite fast in practice. To obtain the actual flow,
 ↪  look at
 *     positive values only.
 * Time: $O(V^2\sqrt E)$
 * Status: Tested on Kattis and SPOJ, and stress-tested
 */

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

    void addEdge(int s, int t, ll cap, ll rcap=0) {
        if (s == t) return;
        g[s].push_back({t, sz(g[t]), 0, cap});
        g[t].push_back({s, sz(g[s])-1, 0, rcap});
    }

    void addFlow(Edge& e, ll f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f; e.c -= f; ec[e.dest] += f;
        back.f -= f; back.c += f; ec[back.dest] -= f;
    }
    ll calc(int s, int t) {
        int v = sz(g); H[s] = v; ec[t] = 1;
        vi co(2*v); co[0] = v-1;
        rep(i,0,v) cur[i] = g[i].data();
        for (Edge& e : g[s]) addFlow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty()) if (!hi--) return -ec[s];
            int u = hs[hi].back(); hs[hi].pop_back();
            while (ec[u] > 0)  // discharge u
                if (cur[u] == g[u].data() + sz(g[u])) {
                    H[u] = 1e9;
                    for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
                        H[u] = H[e.dest]+1, cur[u] = &e;
                    if (++co[H[u]], !--co[hi] && hi < v)
                        rep(i,0,v) if (hi < H[i] && H[i] < v)
                            --co[H[i]], H[i] = v + 1;
                    hi = H[u];
                } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
                    addFlow(*cur[u], min(ec[u], cur[u]->c));
                else ++cur[u];
        }
    }
    bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

Graphs algorthms

Acyclic Ordering

```
using ll = long long;
using dig_adj = vector<vector<ll>>;
using vll = vector<ll>;

void dfsa_aux(dig_adj &d, vll &pred, vll &tvisit, vll &texpl, vll
↪  &verts, ll &time, ll &i, ll v);
/**
 * Algoritmo de Knuth para ordenamiento acíclico. O(n+m). Variación de
 ↪  DFS.
 *
 * Sea D una digráfica y sea x_1, x_2, ..., x_n un ordenamiento de sus
 ↪  vértices.
 * Decimos que el ordenamiento es acíclico si para cada flecha x_ix_j
 ↪  en D, i<j.
 * Existe si y sólo si la digráfica no tiene un ciclo dirigido.
 *
 * Como toda digráfica acíclica tiene fuente y sumidero, la idea del
 ↪  algoritmo
 * es escoger fuentes, "quitarlas" y repetir.
```

```
 *
 * Como este algoritmo es correcto si la digráfica es acíclica, se
 ↪  puede
 * verificar si la digráfica es acíclica al ejecutarlo y revisar si el
 * ordenamiento es acíclico.
 *
 * Este algoritmo es auxiliar al algoritmo de componentes fuertemente
 ↪  conexas.
 */
vll dfsa(ll n, dig_adj &d)
{
    // pred - predecesor || tvisit - tiempo de visita || texpl -
    ↪  tiempo de exploración
    vll pred(n,-1), tvisit(n,0), texpl(n,0), verts(n,-1);
    ll time = 0, i = n;

    for (ll v = 0; v < n; v++)
    {
        if (tvisit[v]) continue;
        dfsa_aux(d, pred, tvisit, texpl, verts, time, i, v);
    }

    /**
     * Hasta aquí tenemos:
     *   lista de predecesores - pred
     *   tiempos de visita de cada vértice - tvisit
     *   tiempo en el que se explora (tras sus hijos) el vértice -
     ↪  texpl
     *   el ordenamiento acíclico - verts
     */
    return verts;
}
// Auxiliar que realiza la búsqueda por profundidad recursivamente
void dfsa_aux(dig_adj &d, vll &pred, vll &tvisit, vll &texpl, vll
↪  &verts, ll &time, ll &i, ll v)
{
    tvisit[v] = ++time;
    for (ll u : d[v])
    {
        if (tvisit[u]) continue;
        pred[u] = v;
        dfsa_aux(d, pred, tvisit, texpl, verts, time, i, u);
    }
    texpl[v] = ++time;
    verts[--i] = v;
}
```

Bellman-Ford

```
struct Edge {
    int a, b, cost;
};

int n, m, v; // #verts, #edges, vert ini
vector<Edge> edges;
const int INF = 1000000000;

// Bellman Ford que sólo nos dice el costo desde v a los demás
void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (;;) {
        bool any = false;

        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    any = true;
                }

        if (!any)
            break;
    }
    // display d, for example, on the screen
}

// Bellman Ford con la ruta desde v hacia t
void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);

    for (;;) {
        bool any = false;
```

```
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    p[e.b] = e.a;
                    any = true;
                }
        if (!any)
            break;
    }

    if (d[t] == INF)
        cout << "No path from " << v << " to " << t << ".";
    else {
        vector<int> path;
        for (int cur = t; cur != -1; cur = p[cur])
            path.push_back(cur);
        reverse(path.begin(), path.end());

        cout << "Path from " << v << " to " << t << ": ";
        for (int u : path)
            cout << u << ' ';
    }
}

// Bellman Ford con detección de ciclos negativos
void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);
    int x;
    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = max(-INF, d[e.a] + e.cost);
                    p[e.b] = e.a;
                    x = e.b;
                }
    }

    if (x == -1)
        cout << "No negative cycle from " << v;
    else {
        int y = x;
        for (int i = 0; i < n; ++i)
            y = p[y];

        vector<int> path;
        for (int cur = y;; cur = p[cur]) {
            path.push_back(cur);
            if (cur == y && path.size() > 1)
                break;
        }
        reverse(path.begin(), path.end());

        cout << "Negative cycle: ";
        for (int u : path)
            cout << u << ' ';
    }
}

////////////////////////////

// Shortest path faster algorithm (promedio O(m), peor caso O(nm))
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;
bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<bool> inqueue(n, false);
    queue<int> q;

    d[s] = 0;
    q.push(s);
    inqueue[s] = true;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inqueue[v] = false;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;
```

```
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inqueue[to]) {
                    q.push(to);
                    inqueue[to] = true;
                    cnt[to]++;
                    if (cnt[to] > n)
                        return false;  // negative cycle
                }
            }
        }
    }
    return true;
}
```

Cycle Detection Directed

```
int n;
vector<vector<int>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

Cycle Detection Undirected

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing vertex and its parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // skipping edge to parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
```

```
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

Dijkstra

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>> q;
    q.push({0, s});
    while (!q.empty()) {
        int v = q.top().second;
        int d_v = q.top().first;
        q.pop();
        if (d_v != d[v])
            continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push({d[to], to});
            }
        }
    }
}
```

Euler tour

```
const int N = 2e5 + 1;
int timer = 0;
int start[N];
int endd[N];
ll arreglo_segtree[N];
ll values[N];
vector<int> graph[N];
void euler_tour(int u, int p)
{
  start[u] = timer++;
  for (int v : graph[u])
    if (v != p)
      euler_tour(v, u);
  endd[u] = timer;
}

int main()
{
  int n, q; cin >> n >> q;
  for (int i = 0; i < n; i++)
```

```
  cin >> values[i];

// Gráfica con nodos 0-indexados
for (int i = 0; i < n - 1; i++)
{
  int u, v; cin >> u >> v;
  u--;
  v--;
  graph[u].push_back(v);
  graph[v].push_back(u);
}

// Empezamos el euler tour con la raiz
int root = 0;
euler_tour(root, -1);
// Para este punto start[i] y end[i] tienen el tiempo en el que se
↪  visito y se terminó de visitar el vertice i en el árbol
// end[i] - start[i] = Tamaaño del subarbol de i (incluyendolo)

// Construimos un arreglo con los valores de los nodos, con el orden
↪  dado por start, que hará que los nodos estén contiguos y podamos
↪  hacer uso de un SegmentTree
for (int i = 0; i < n; i++)
  arreglo_segtree[start[i]] = values[i];

SegmentTree<ll> st(arreglo_segtree, n);
// start[i] = indice desde donde comienza el subarbol de i
↪  (incluyendolo) en arreglo_segtree
// endd[i] - 1 = incide donde termina el subarbol de i en
↪  arreglo_segtree (es importante restarle uno)

while (q--)
{
  int op; cin >> op;
  int node; cin >> node;
  node--;

  int l = start[node];
  int r = endd[node] - 1;

  if (op == 1)
  {
    ll val;
    cin >> val;
      // l es la posición donde está el nodo en arreglo_segtree
      st.update(1, 0, n - 1, l, val);
    }
    else
    {
      // En caso de que no se incluya a nodo en el subarbol
      // puede pasar que el subarbol sea vacío
      if (r < l)
      {
        cout << 0 << endl;
        continue;
      }
      cout << st.query(1, 0, n - 1, l, r) << endl;
    }
  }
  return 0;
}
```

Floyd Warshall

```
const int INF = 1e9;
int n;

vector<vector<int>> d(n, vector<int>(n, INF));
// Inicializamos la matriz de distancias:
// - d[i][j] = INF si no hay un camino directo conocido de i a j
// - d[i][i] = 0 para todo i (la distancia de un nodo a sí mismo es 0)
// - d[i][j] = peso de la arista de i a j, si existe una arista directa

// Floyd-Warshall: encuentra distancias mínimas entre todos los pares
↪  (i, j)
for (int k = 0; k < n; ++k) {
    // Usamos el nodo k como intermediario
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            // Solo procesamos si los caminos i -> k y k -> j existen
            if (d[i][k] < INF && d[k][j] < INF)
                // Verificamos si ir de i a j pasando por k es más
                ↪  barato
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

Fuertemente Conexa Kosaraju-Sharir

```
// NOTE: Hace uso de Acyclic Ordering (dfsa)

using ll = long long;
using dig_adj = vector<vector<ll>>;
using vll = vector<ll>;

void sca_aux(dig_adj &conv, vll &pred, vll &tvisit, vll &texpl, ll
↪  &time, vll &componente, ll comp_idx, ll v);
/**
 * Componentes fuertemente conexas (Kosaraju-Sharir). O(n+m).
 ↪  Variación de DFS.
 *
 * Además, construimos la gráfica de bloques fuertes pero se indica en
 ↪  un
 * comentario donde podemos deternernos si sólo queremos las
 ↪  componentes.
 * A veces a esto se le llama la condensación de la digráfica.
 *
 * Aquí estoy realizando una pequeña modificación para el problema:
 ↪  construyo
 * la digráfica de bloques fuertes con el costo acumulado de todos los
 ↪  vértices
 * en cada bloque (pues todo bloque únicamente puede ser elegido todo
 ↪  junto).
 *
 * Recuerde que la digráfica conversa es la que tiene las flechas
 ↪  invertidas
 */
dig_adj sca(ll n, dig_adj &d, dig_adj &d_converse)
{
    vll ord_aciclico = dfsa(n, d);

    // pred - predecesor || tvisit - tiempo de visita || texpl -
    ↪  tiempo de exploración
    vll pred(n,-1), tvisit(n,0), texpl(n,0);
    vll componente(n,-1); // a cuál componente pertenece cada vértice
    ll time = 0, componente_idx = 0;

    // La consulta de los vértices debe ser sobre el orden acíclico
    for (ll v : ord_aciclico)
    {
        if (tvisit[v]) continue;
        sca_aux(d_converse, pred, tvisit, texpl, time, componente,
        ↪  componente_idx++, v);
    }

    /**
     * Hasta aquí tenemos:
     *   lista de predecesores - pred
     *   tiempos de visita de cada vértice - tvisit
     *   tiempo en el que se explora (tras sus hijos) el vértice -
     ↪  texpl
     *   a qué componente pertenece cada vértice - componente
     *   cuántas componentes hay - componente_idx
     */

    dig_adj bloques(componente_idx);
    // Revisamos todas las aristas y agregamos las que unen componentes
    for (ll u = 0; u < n; u++)
    for (ll v : d[u])
    {
        ll cu = componente[u];
        ll cv = componente[v];
        if (cu == cv) continue;
        bloques.m++;
        bloques[cu].push_back(cv);
    }

    return bloques;
}
// Auxiliar que realiza la búsqueda por profundidad recursivamente
void sca_aux(dig_adj &conv, vll &pred, vll &tvisit, vll &texpl, ll
↪  &time, vll &componente, ll comp_idx, ll v)
{
    tvisit[v] = ++time;
    componente[v] = comp_idx;
    for (ll u : conv[v])
    {
        if (tvisit[u]) continue;
        pred[u] = v;
        sca_aux(conv, pred, tvisit, texpl, time, componente, comp_idx,
        ↪  u);
    }
    texpl[v] = ++time;
}
```

Havel-hakin

```python
import sys

def todos_son_ceros(t):
    for d in t:
        if d != 0:
            return False
    return True

def esGrafica(grados):
    if sum(grados) % 2 != 0:
        return False

    grados = sorted(grados, reverse=True)

    if todos_son_ceros(grados):
        return True

    d = grados[0]
    restantes = list(grados[1:])

    if d > len(restantes):
        return False

    for i in range(d):
        restantes[i] -= 1
        if restantes[i] < 0:
            return False

    return esGrafica(restantes)

def main():
    data = sys.stdin.read().strip().split()
    grad = list(map(int, data))
    print(esGrafica(grad))

if __name__ == "__main__":
    main()
```

Is Bipartite

```cpp
int n;
vector<vector<int>> adj;

vector<int> side(n, -1); // side[i] = 0 o 1 si ya fue asignado, -1 si
↪  no ha sido visitado
bool is_bipartite = true;

queue<int> q;
// Recorremos todos los componentes del grafo
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        // Si el nodo aún no ha sido visitado, iniciamos BFS desde él
        q.push(st);
        side[st] = 0; // Le asignamos el color 0

        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    // Si el vecino no ha sido visitado, lo coloreamos
                    ↪  con el color opuesto
                    side[u] = side[v] ^ 1; // Alternamos entre 0 y 1
                    q.push(u);
                } else {
                    // Si el vecino ya está coloreado y tiene el mismo
                    ↪  color, no es bipartito
                    if (side[u] == side[v])
                        is_bipartite = false;
                }
            }
        }
    }
}
cout << (is_bipartite ? "YES" : "NO") << endl;
```

Tarjan

```cpp
#include <iostream>
#include <vector>
#include <set>
using namespace std;
```

```cpp
const int MAXN = 100005;  // ajusta según el tamaño de tu grafo
vector<int> grafo[MAXN];
bool visitado[MAXN];
int tin[MAXN], low[MAXN];
int timer = 0;
set<int> puntos_articulacion;  // para evitar repetidos

void dfs(int u, int padre = -1) {
    visitado[u] = true;
    tin[u] = low[u] = timer++;
    int hijos = 0;

    for (int v : grafo[u]) {
        if (v == padre) continue;

        if (visitado[v]) {
            // v ya fue visitado: actualiza low
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u);
            low[u] = min(low[u], low[v]);
            if (low[v] >= tin[u] && padre != -1) {
                puntos_articulacion.insert(u);
            }
            ++hijos;
        }
    }

    // Caso especial: si u es la raíz y tiene más de un hijo
    if (padre == -1 && hijos > 1) {
        puntos_articulacion.insert(u);
    }
}

int main() {
    int n, m;
    cin >> n >> m; // número de nodos y aristas
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        grafo[u].push_back(v);
        grafo[v].push_back(u);
    }

    for (int i = 0; i < n; ++i) {
        if (!visitado[i]) dfs(i);
    }

    cout << "Puntos de articulación:\n";
    for (int p : puntos_articulacion) {
        cout << p << "\n";
    }

    return 0;
}
```

Tarjan puentes

```cpp
#include <iostream>
#include <vector>
using namespace std;

const int MAXN = 100005;
vector<int> grafo[MAXN];
bool visitado[MAXN];
int tin[MAXN], low[MAXN];
int timer = 0;
vector<pair<int, int>> puentes;

void dfs(int u, int padre = -1) {
    visitado[u] = true;
    tin[u] = low[u] = timer++;

    for (int v : grafo[u]) {
        if (v == padre) continue;

        if (visitado[v]) {
            // back edge
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u);
            low[u] = min(low[u], low[v]);

            if (low[v] > tin[u]) {
                // u-v es un puente
                puentes.push_back({u, v});
```

```
            }
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m; // número de nodos y aristas

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        grafo[u].push_back(v);
        grafo[v].push_back(u); // no dirigido
    }

    for (int i = 0; i < n; ++i) {
        if (!visitado[i]) dfs(i);
    }

    cout << "Puentes encontrados:\n";
    for (auto [u, v] : puentes) {
        cout << u << " - " << v << "\n";
    }

    return 0;
}
```

Topological Sort

```
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

// Se asume que la gráfica es acíclica
void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

Weighted Bipartite Matching (Hungarian)

```
/**
 * Author: Benjamin Qi, chilli
 * Date: 2020-04-04
 * License: CC0
 * Source: https://github.com/bqi343/USACO
 * Description: Given a weighted bipartite graph, matches every node on
 * the left with a node on the right such that no
 * nodes are in two matchings and the sum of the edge weights is
 ↪  minimal. Takes
 * cost[N][M], where cost[i][j] = cost for L[i] to be matched with
 ↪  R[j] and
 * returns (min cost, match), where L[i] is matched with
 * R[match[i]]. Negate costs for max cost. Requires $N \le M$.
 * Time: O(N^2M)
 * Status: Tested on kattis:cordonbleu, stress-tested
 */
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

pair<int, vi> hungarian(const vector<vi> &a) {
    if (a.empty()) return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i,1,n) {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
```

```
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j,1,m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            rep(j,0,m) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
    }
    rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans}; // min cost
}
```

Trees

Centroid Decomposition

```
// A partir de un árbol como gráfica, construye en O(nlogn) un árbol
↪  enraizado
// balanceado (de altura O(logn)) dividiéndolo recursivamente en
↪  centroides

using graf_t = vector<vector<int>>;

struct ctree_n
{
    int id, parent;
    vector<int> children;
    ctree_n(int id_, int parent_ = -1) : id(id_), parent(parent_),
    ↪  children({}) {}
};

struct ctree
{
    vector<int> size; // size of node[i] subtree
    vector<ctree_n> node;
    int root;

    ctree(const graf_t& graf)
        : size(graf.size(),0)
    {
        int n = graf.size();
        node.reserve(n);
        vector<bool> visited(n,false);
        for (int i = 0; i < n; i++) node.emplace_back(i);
        decompose(0, -1, visited, graf);
    }

private:
    void decompose(int u, int parent, vector<bool>& visited, const
    ↪  graf_t& graf)
    {
        vector<bool> viscp1(visited), viscp2(visited);
        int total = subtree_size(u, -1, viscp1, graf);
        int centroid = find_centroid(u, -1, total, viscp2, graf);
        visited[centroid] = true;
        node[centroid].parent = parent;
        if (parent == -1) node[centroid].parent = root = centroid;
        else node[parent].children.push_back(centroid);
        for (int v : graf[centroid])
            if (!visited[v]) decompose(v, centroid, visited, graf);
    }

    int subtree_size(int u, int p, vector<bool>& visited, const
    ↪  graf_t& graf)
    {
        size[u] = 1;
        for (int v : graf[u])
            if (v != p && !visited[v])
                size[u] += subtree_size(v, u, visited, graf);
        return size[u];
    }

    int find_centroid(int u, int p, int total, vector<bool>& visited,
    ↪  const graf_t& graf)
    {
```

```
        for (int v : graf[u])
            if (v != p and !visited[v] and size[v] > total/2)
                return find_centroid(v, u, total, visited, graf);
        return u;
    }
};
```

Farthest node for each node

```
// dist[0][i] = distancia del nodo 'a' hacia todos los nodos i
// dist[1][i] = distancia del nodo 'b'' hacai todos los nodos i
int dist[2][N];
vector<int> graph[N];

// f_n = nodo más lejano
int dfs(int u, int p, int d, int i)
{
  dist[i][u] = d;
  int f_n = -1;
  for (int v : graph[u])
  {
    if (v != p)
    {
      int x = dfs(v, u, d + 1, i);
      if (f_n == -1 || dist[i][x] > dist[i][f_n]) f_n = x;
    }
  }
  return f_n == -1 ? u : f_n;
}

void calculate_diameter()
{
  int n; cin >> n;
  for (int i = 0; i < n - 1; i++)
  {
    int a, b;
    cin >> a >> b;
    graph[a-1].push_back(b-1);
    graph[b-1].push_back(a-1);
  }
  // Encontramos el nodo 'a' haciendo un dfs desde un nodo arbitrario
  int a = dfs(0, 0, 0, 0);
  // Encontramos el nodo 'b' haciendo un dfs desde 'a', y su vez
  ↪ calculamos la distancia de 'a' hacia todos los demás nodos
  int b = dfs(a, a, 0, 0);

  // Finalmente calculamos la distncia del nodo 'b' hacia todos los
  ↪ demas
  dfs(b, b, 0, 1);

  // dist(a, b) = Longitud del diametro del árbol
  // max(dist[0][u], dist[1][u]) = distancia de u hacia su nodo más
  ↪ lejano
}
```

Heavy light decomposition

```
// En un árbol enraizado, marcamos a una única arista hacia un hijo
↪ como pesada
// si cuenta con la mayor cantidad de vértices en su subárbol. Esto
↪ nos forma
// trayectorias pesadas, sobre las cuales podemos agregar
↪ segment-trees de
// prefijos para hacer updates/queries de trayectorias, pues se puede
↪ mostrar
// que cualquier trayectoria cuenta con a lo más O(logn) de
↪ trayectorias
// pesadas. Las aristas no-pesadas les llamamos ligeras.
// La descomposición toma O(n).

// En varios casos se puede usar un segment tree sobre el euler tour
↪ en lugar
// de hacer esto.

// parent[v] es el padre de v, depth[v] es la profundidad en v,
↪ heavy[v] indica
// al hijo con el cual <v,heavy[v]> es arista pesada, head[v] es donde
↪ termina
// el heavy path de v, pos[v] es la posición en el segment-tree de
↪ dicho heavy
// path completo. En decompose se llena head y pos.
vector<int> parent, depth, heavy, head, pos;
int cur_pos;
```

```cpp
// esto se llama desde init, no directamente
int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

// esto se llama desde init, no directamente
void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

// se asume v=0 raíz
void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    cur_pos = 0;

    dfs(0, adj);
    decompose(0, 0, adj);
}

// max-query en este ejemplo
// Encontramos lca(a, b)=L, hacemos la query del segment tree a->L y
↪ L->b
// Como hay un segment tree por heavy path y hay logn heavy paths,
↪ O(log^2 n)
// |- (pero si precalculas los prefijos es logn)
// TODO: implementar segment_tree_query(-, -). En lugar de implementar
↪ logn
//       segment-trees, se puede implementar uno solo con segmentos
↪ disjuntos
//       por cada heavy path
int query(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        int cur_heavy_path_max = segment_tree_query(pos[head[b]],
        ↪ pos[b]);
        res = max(res, cur_heavy_path_max);
    }
    if (depth[a] > depth[b])
        swap(a, b);
    int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}
```

LCA

```cpp
// Puede ser útil recordar que dist(a,b) = depth(a) + depth(b) -
↪ 2*depth(lca(a,b))
int n, l;                 // n = número de nodos, l = log2(n)
vector<vector<int>> adj; // Lista de adyacencia del árbol

int timer;               // Temporizador para asignar tin y tout
vector<int> tin, tout;   // Tiempo de entrada y salida en DFS
vector<vector<int>> up; // up[v][i] = 2^i-ésimo ancestro de v

// DFS para preprocesar ancestros y tiempos
void dfs(int v, int p)
{
  tin[v] = ++timer; // Al entrar al nodo v
  up[v][0] = p;     // 2^0-ésimo ancestro es el padre directo

  // Computar los 2^i-ésimos ancestros
  for (int i = 1; i <= l; ++i)
```

```
    up[v][i] = up[up[v][i - 1]][i - 1];

  // Continuar DFS con los hijos
  for (int u : adj[v])
  {
    if (u != p)
      dfs(u, v);
  }

  tout[v] = ++timer; // Al salir del nodo v
}

// Verifica si u es ancestro de v usando tin y tout
bool is_ancestor(int u, int v)
{
  return tin[u] <= tin[v] && tout[u] >= tout[v];
}

// Devuelve el Lowest Common Ancestor de u y v
int lca(int u, int v)
{
  // Si uno es ancestro del otro, es el LCA
  if (is_ancestor(u, v))
    return u;
  if (is_ancestor(v, u))
    return v;

  // Subimos a u hasta que su ancestro está justo por debajo del LCA
  for (int i = l; i >= 0; --i)
  {
    if (!is_ancestor(up[u][i], v))
      u = up[u][i];
  }

  return up[u][0]; // El padre de u en este punto es el LCA
}

// Preprocesamiento inicial para Binary Lifting
void preprocess(int root)
{
  tin.resize(n);
  tout.resize(n);
  timer = 0;
  l = ceil(log2(n)); // Límite de profundidad de los ancestros
  up.assign(n, vector<int>(l + 1));
  dfs(root, root); // Empezamos el DFS desde la raíz
}

int main()
{
  n = 7;
  adj.resize(n);
  adj[0] = {1, 2};
  adj[1] = {0, 3, 4};
  adj[2] = {0, 5, 6};
  adj[3] = {1};
  adj[4] = {1};
  adj[5] = {2};
  adj[6] = {2};

  preprocess(0); // Raíz del árbol en 0

  cout << lca(3, 4) << endl; // Salida esperada: 1
  cout << lca(5, 6) << endl; // Salida esperada: 2
  cout << lca(3, 5) << endl; // Salida esperada: 0
}
```

Strings

Aho-Corasik

```
const int M = 26;
struct node
{
  vector<int> child;
  int p = -1;
  char c = 0;
  int suffixLink = -1, endLink = -1;
  int id = -1;
  node(int p = -1, char c = 0) : p(p), c(c)
  {
    child.resize(M, -1);
  }
};
```

```
struct AhoCorasick
{
  vector<node> t;
  vector<int> lengths; // Longitud de las palabras en el trie
  vector<int> occ;     // Ocurrencias por patrón
  int wordCount = 0;
  AhoCorasick()
  {
    t.emplace_back();
  }

  // Regresa el ID que se le asigno a la palabra agregada (0-indexado)
  int add(const string &s)
  {
    int u = 0; // Raíz temporal
    for (char c : s)
    {
      if (t[u].child[c - 'a'] == -1)
      {
        t[u].child[c - 'a'] = t.size();
        t.emplace_back(u, c);
      }
      u = t[u].child[c - 'a'];
    }
    if (t[u].id == -1) // Por si se trata de agregra una palabra que
    ↪  ya estaba
      t[u].id = wordCount++;
    lengths.push_back(s.size());
    return t[u].id;
  }

  void link(int u)
  {
    if (u == 0) // Si eres la raíz
    {
      t[u].suffixLink = 0;
      t[u].endLink = 0;
      return;
    }
    if (t[u].p == 0) // Si tu padre es la raíz
    {
      t[u].suffixLink = 0;
      if (t[u].id != -1)
        t[u].endLink = u;
      else
        t[u].endLink = t[t[u].suffixLink].endLink;
      return;
    }

    int v = t[t[u].p].suffixLink;
    char c = t[u].c;
    while (true)
    {
      if (t[v].child[c - 'a'] != -1)
      {
        t[u].suffixLink = t[v].child[c - 'a'];
        break;
      }
      if (v == 0)
      {
        t[u].suffixLink = 0;
        break;
      }
      v = t[v].suffixLink;
    }
    if (t[u].id != -1) // Si eres una palabra
      t[u].endLink = u;
    else
      t[u].endLink = t[t[u].suffixLink].endLink;
  }

  // BFS para construir los suffix links y end lins. Llamar a esto
  // cuando ya hayas metido todas strings del diccionario al trie
  void build()
  {
    queue<int> Q;
    Q.push(0);
    while (!Q.empty())
    {
      int u = Q.front();
      Q.pop();
      link(u);
      for (int v = 0; v < M; ++v)
        if (t[u].child[v] != -1)
          Q.push(t[u].child[v]);
    }
    occ.assign(wordCount, 0);
  }
```

```
  int match(const string &text)
  {
    int u = 0;
    int ans = 0;
    for (int j = 0; j < text.size(); ++j)
    {
      int i = text[j] - 'a';
      while (true)
      {
        if (t[u].child[i] != -1)
        {
          u = t[u].child[i];
          break;
        }
        if (u == 0)
          break;
        u = t[u].suffixLink;
      }

      // Temporal de u para moverse por el endLink para el caso
      // en el que haya sufijos propios de lo que llevamos visitado
      // que su vez sean palabras
      int v = u;
      while (true)
      {
        v = t[v].endLink;
        if (v == 0)
          break;
        ++ans;
        // int idx = j + 1 - lengths[t[v].id];
        // cout << "Found word #" << t[v].id << " at position " << idx
        ↪  << "\n";
        occ[t[v].id]++;
        v = t[v].suffixLink; // Para eviar el caso en el que el nodo
        ↪  de v es el de una palabra
      }
    }
    return ans;
  }
};

int main()
{
  AhoCorasick ac;
  string text;
  cin >> text;
  int n;
  cin >> n;
  vector<int> ids(n); // Por si se agrega varias veces la misma palabra
  for (int i = 0; i < n; i++)
  {
    string word;
    cin >> word;
    ids[i] = ac.add(word);
  }
  ac.build();
  ac.match(text);
  for (int i = 0; i < n; i++)
    cout << ac.occ[ids[i]] << endl;
  return 0;
}
```

Counting Substrings

```
/*
Given a string s and q queries, each query is a string t
For each request you need to count how many times the string t
occurs as a substring in s.
O(t log(n)) where n = s.size() for each query
*/

int lwb(vi &SA, string &s, string &t)
{
  int l = 0;
  int r = SA.size() - 1;

  int res = SA.size();
  while (l <= r)
  {
    int m = l + (r - l) / 2;
    if (s.substr(SA[m], t.size()) >= t)
    {
      res = m;
      r = m - 1;
```

```
    }
    else
    {
      l = m + 1;
    }
  }
  return res;
}

int upb(vi &SA, string &s, string &t)
{
  int l = 0;
  int r = SA.size() - 1;
  int res = SA.size();
  while (l <= r)
  {
    int m = l + (r - l) / 2;
    if (s.substr(SA[m], t.size()) > t)
    {
      res = m;
      r = m - 1;
    }
    else
    {
      l = m + 1;
    }
  }
  return res;
}

int main()
{
  ios::sync_with_stdio(0);
  cin.tie(nullptr);

  string s;
  cin >> s;
  s += "$";

  vi SA = Suffix_Array(s);

  int q;
  cin >> q;
  while (q--)
  {
    string t;
    cin >> t;
    cout << upb(SA, s, t) - lwb(SA, s, t) << endl;
  }
  return 0;
}
```

Finding Borders

```
/*
A border of a string is a prefix that is also a suffix of the string
↪  but not the whole string. For example, the borders of abcababcab
↪  are ab and abcab.
Your task is to find all border lengths of a given string:
*/
void Finding_Borders(string& s){
  vector<int> pi = prefix_function(s);
  // Empezamos con el borde más grande
  int j = pi[s.size() - 1];
  vector<int> res;
  while (j > 0)
  {
    res.push_back(j);
    // El siguiente borde más grande
    j = pi[j - 1];
  }
  for (int i = res.size() - 1; i >= 0; i--)
    cout << res[i] << " ";
}
```

Finding Periods

```
/*
A period of a string is a prefix that can be used to generate the
↪  whole string by repeating the prefix. The last repetition may be
↪  partial. For example, the periods of abcabca are abc, abcabc and
↪  abcabca.
Find all period lengths of a string:
*/
void Finding_Periods(string& s){
```

```
  int n = s.size();
  vector<int> z = Z_function(s);

  for (int i = 0; i < n; i++)
    if (z[i] == n - i)
      cout << i << " ";

  cout << n << endl;
}
```

Hashing

```
struct Hash
{
  ll P = 1777771; // Primo para sacar el hash
  ll MOD[2], PI[2];
  vector<ll> h[2]; // Contiene los prefix sum de los hahses
  vector<ll> pi[2]; // pi[k][i] contiene PI[k] elevado a la i
  Hash(string &s)
  {
    MOD[0] = 999727999;
    MOD[1] = 1070777777;
    PI[0] = 325255434; // Inverso multiplicativo de P modulo MOD[0]
    PI[1] = 10018302; // Inverso multiplicativo de P modulo MOD[1]
    for (int k = 0; k < 2; k++)
      h[k].resize(s.size() + 1), pi[k].resize(s.size() + 1);
    for (int k = 0; k < 2; k++)
    {
      h[k][0] = 0;
      pi[k][0] = 1;
      ll p = 1;
      for (int i = 1; i < s.size() + 1; i++)
      {
        h[k][i] = (h[k][i - 1] + p * s[i - 1]) % MOD[k];
        pi[k][i] = (1LL * pi[k][i - 1] * PI[k]) % MOD[k];
        p = (p * P) % MOD[k];
      }
    }
  }
  // Returns the hash of substring [s,e)
  ll get(int s, int e)
  {
    ll h0 = (h[0][e] - h[0][s] + MOD[0]) % MOD[0];
    h0 = (1LL * h0 * pi[0][s]) % MOD[0];
    ll h1 = (h[1][e] - h[1][s] + MOD[1]) % MOD[1];
    h1 = (1LL * h1 * pi[1][s]) % MOD[1];
    return (h0 << 32) | h1;
  }
};

// O(n²)
int Distinct_Substrings(string& s){
  int n = s.size();
  Hash s_hash(s);
  unordered_set<ll> aux;
  for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++)
      aux.insert(s_hash.get(i, j + 1));

 return aux.size();
}

// Nos dice si la substring s[l2...r2] es menor que la string
↪  s[l1...r1] en
// tiempo log_2(n)
bool less_str(string &s, Hash &hash, int l1, int r1, int l2, int r2)
{
  int l = 0;
  int r = s.size() - 1;

  // Busqueda binaria en la longitud máxima del prefijo que es igual
  ↪  en ambas strings
  int last = 0;
  while (l <= r)
  {
    int m = l + (r - l) / 2;
    int h1 = hash.get(l1, l1 + m);
    int h2 = hash.get(l2, l2 + m);

    if (h1 == h2)
    {
      last = m;
      l = m + 1;
    }
    else
    {
      r = m - 1;
```

```
    }
  }

  // s[l1...l1+last) == s[l2...l2+last)
  // Si last < s.size() && s[l2+last] < s[l1+last] entonces
  // s[l2...r2] es menor que la string s[l1...r1]
  return last < s.size() && s[l2 + last] < s[l1 + last];
}
```

Hashing Strings as vectors

```
vector<ll> P = {325255434, 10018302};
vector<ll> M = {999727999, 1070777777};

/*
  Para tratar todas las permutaciones de una misma cadena como
  ↪ equivalentes,
  se puede representar la cadena mediante su vector de frecuencias.
  Por ejemplo, la cadena "abacb" se representa como:
    f = {2, 2, 1, 0, 0, ...}
          a  b  c  d  e ...
  donde f[i] indica cuántas veces aparece el carácter en la posición i
  ↪ del alfabeto en la string dada.

  Para comparar estas representaciones de forma eficiente, se puede
  ↪ calcular
  un hash del vector de frecuencias. Sea:
    - f el vector de frecuencias de una cadena,
    - |A| el tamaño del alfabeto (por ejemplo, 26 para letras
    ↪ minúsculas),
    - b y M constantes enteras elegidas para el cálculo del hash,

  entonces definimos el hash como:
    hash(f) = (f[0] * b^0 + f[1] * b^1 + ... + f[|A|-1] * b^|A|-1) mod
    ↪ M

  Nota: Si ya se ha calculado el hash del vector correspondiente a la
  ↪ subcadena s[L...R] entonces calcular el hash del vector de la
  ↪ subcadena definida por
  s[L...R+1] implica:
    hash(S[L...R+1]) = ( hash(S[L...R]) + b^(pos(S[R+1])) ) mod M
  donde pos(S[R+1]) es la posición del caracter S[R+1] en el alfabeto.
*/
void Hashing_Strigns_as_Vectors()
{
  string s;
  int sigma = 27; // Tamaño del alfabeto
  int n = s.size();

  vector<vector<ll>> pot(2, vector<ll>(sigma, 0));
  pot[0][0] = pot[1][0] = 1;

  for (int i = 1; i < sigma; i++)
  {
    pot[0][i] = pot[0][i - 1] * P[0] % M[0];
    pot[1][i] = pot[1][i - 1] * P[1] % M[1];
  }

  // Calcula el hash de todos los vectores de frecuencias de
  // todas las substrings de s
  for (int i = 0; i < n; i++)
  {
    ll hash1 = 0;
    ll hash2 = 0;
    for (int j = i; j < n; j++)
    {
      int pos_c = s[j] - 'a';
      hash1 += pot[0][pos_c] % M[0];
      hash2 += pot[1][pos_c] % M[1];

      ll final_hash = (hash1 << 32) | hash2;
    }
  }
}
```

KMP

```
vector<int> KMP(const string &text, const string &pat)
{
  int n = text.size();
  vector<int> pi = prefix_function(pat);
  int j = pi[0];
  vector<int> pi_text(n, 0);
  for (int i = 0; i < n; i++)
```

```
  {
    while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
    if (text[i] == pat[j]) j++;
    pi_text[i] = j;
    if (j == pat.size()) j = pi[j - 1];
  }
  // Las ocurrencias de pat.size() en pi_text
  // son las apariciones de pat en text
  return pi_text;
}
```

Longest Common Substring

```
// Nos dice si i pertenece a la string a y j a la string b o viceversa
bool distintos(int n1, int n, int i, int j)
{
  return (i >= 0 && i < n1 && j >= n1 + 1 && j < n) || (j >= 0 && j <
  ↪  n1 && i >= n1 + 1 && i < n);
}

string LCS()
{
  string a;
  cin >> a;
  int n1 = a.size();

  string b;
  cin >> b;
  int n2 = b.size();

  string s = a + "$" + b + "#";

  pair<vi, vi> result = Suffix_Array_and_LCP(s);
  vi SA = result.first;
  vi LCP = result.second;

  int res = 0;
  int maximo = INT_MIN;
  for (int i = 1; i < s.size(); i++)
  {
    if (distintos(n1, s.size(), SA[i], SA[i - 1]))
    {
      if (LCP[SA[i]] > maximo)
      {
        maximo = LCP[SA[i]];
        res = SA[i];
      }
    }
  }
  return s.substr(res, LCP[res]);
}
```

Manacher

```
// Palíndromos impares centrados en cada posición
vector<int> manacher_odd(const string& s) {
    int n = s.size();
    vector<int> odd(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; ++i) {
        int k = (i > r ? 1 : min(odd[l + r - i], r - i + 1));
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        odd[i] = k--;
        if (i + k > r) {
            l = i - k;
            r = i + k;
        }
    }
    return odd;
}
// Palíndromos pares centrados entre posiciones
vector<int> manacher_even(const string& s) {
    int n = s.size();
    vector<int> even(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; ++i) {
        int k = (i > r ? 0 : min(even[l + r - i + 1], r - i + 1));
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i +
        ↪  k]) k++;
        even[i] = k--;
        if (i + k > r) {
            l = i - k - 1;
            r = i + k;
        }
    }
```

```
    return even;
}

// Función para contar todos los palíndromos
int count_palindromes(const string& s) {
    vector<int> odd = manacher_odd(s);
    vector<int> even = manacher_even(s);
    int total_palindromes = 0;
    // Contar los palíndromos impares
    for (int i = 0; i < s.size(); ++i) {
        total_palindromes += odd[i];  // Cada odd[i] indica cuántos
        ↪  palíndromos impares centrados en i
    }
    // Contar los palíndromos pares
    for (int i = 0; i < s.size(); ++i) {
        total_palindromes += even[i]; // Cada even[i] indica cuántos
        ↪  palíndromos pares centrados entre i-1 e i
    }
    return total_palindromes;
}

int main() {
    string s = "abacaba";
    int total = count_palindromes(s);
    cout << "Total de palíndromos en la cadena: " << total << endl;
    return 0;
}
```

Number of Balanced Substrings

```
// Cuenta el número de subcadenas que están en 1-Dyck en O(n)
// (subcadenas que están en el leng. de paréntesis balanceados)

ll aux_subdyck(const vector<int>& to_open, int start, int end) //
↪  auxiliar
{
    ll streak = 0, acc = 0; // acc es la cuenta, streak es la racha de
                            // paréntesis adyacentes en el mismo nivel
    for (int i = end-1; i >= start; --i)
    {
        int op = to_open[i];
        if (op < 0) { acc += (streak*(streak+1))>>1; streak = 0; }
        else { streak++; acc += aux_subdyck(to_open, op+1, i); i = op;
        ↪  }
    }
    acc += (streak*(streak+1))>>1;
    return acc;
}

ll cnt_subdyck(const string& s, char opensymb, char closesymb) // O(n)
↪  dp
{
    int n = s.length();
    vector<int> opens; opens.reserve(n); // pila de idx de parens
    ↪  abiertos
    vector<int> to_open(n,-1); // dónde se encuentra la pareja '(' de
    ↪  cada ')'
    for (int i = 0; i < n; i++)
    {
        if (s[i] == opensymb)
            opens.push_back(i);
        else if (s[i] == closesymb and opens.size())
        {
            to_open[i] = opens[opens.size()-1];
            opens.pop_back();
        }
    }
    return aux_subdyck(to_open, 0, n); // cuenta por niveles de parens
}
```

Number of Distinct Substrings

```
ll Distinct_Substrings()
{
  string s;
  cin >> s;
  s += "$";

  pair<vi, vi> result = Suffix_Array_and_LCP(s);
  vi SA = result.first;
  vi LCP = result.second;

  ll sum = 0;
  ll n = s.size() - 1; // Para no contar al '$' agregado
```

```
  for (int i = 0; i < n; i++)
    sum += LCP[i];

  ll number_of_substrings = (n * (n + 1)) / 2;
  return number_of_substrings - sum;
}
```

Occurrences Of Each Prefixes

```
vector<int> occurrences_of_each_prefixes(const string& s){
  int n = s.size();
  vector<int> pref = prefix_function(s);
  vector<int> occ(n + 1);
  for (int i = 0; i < n; i++)
    occ[pref[i]]++;
  for (int i = n - 1; i > 0; i--)
    occ[pref[i - 1]] += occ[i];
  for (int i = 0; i <= n; i++)
    occ[i]++;
  return occ;
}
```

Prefix Function

```
/*
pi[i] = Longitud del prefijo propio más grande
en la substring s[0...i] que también es sufijo de
esa misma substring. Por definición pi[0] = 0.

s =  b a b a b a c b b a
pi = 0 0 1 2 3 4 0 1 1 2
    ----------------------
     0 1 2 3 4 5 6 7 8 9
*/
vector<int> prefix_function(const string &s)
{
  int n = s.size();
  vector<int> pi(n, 0);
  int j = 0;
  for (int i = 1; i < n; i++)
  {
    while (j > 0 && s[i] != s[j]) j = pi[j - 1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
  }
  return pi;
}

/*
A border of a string is a prefix that is also a suffix of the string
↪  but not the whole string. For example, the borders of abcababcab
↪  are ab and abcab.
Your task is to find all border lengths of a given string:
*/
void Finding_Borders(string& s){
  vector<int> pi = prefix_fun(s);
  // Empezamos con el borde más grande
  int j = pi[s.size() - 1];
  vector<int> res;
  while (j > 0)
  {
    res.push_back(j);
    // El siguiente borde más grande
    j = pi[j - 1];
  }
  for (int i = res.size() - 1; i >= 0; i--)
    cout << res[i] << " ";
}
```

Repeating Substring

```
/*
A repeating substring is a substring that occurs in two (or more)
↪  locations in the string. Your task is to find the longest
↪  repeating substring in a given string.
*/
string Repeating_Substring()
{
  string s;
  cin >> s;
  s += "$";
  pair<vi, vi> result = Suffix_Array_and_LCP(s);
  vi SA = result.first;
  vi LCP = result.second;
```

```
  int ans = 0;
  for (int i = 0; i < s.size(); i++)
    ans = max(ans, LCP[SA[i]]);

  if (ans == 0)
    return "";

  for (int i = 0; i < s.size(); i++)
    if (ans == LCP[SA[i]])
      return s.substr(SA[i], LCP[SA[i]]);

  return "";
}
```

Suffix Array and LCP

```
/*
Suffix array (SA):
Let s be a string of length n.The i-th suffix of s is the substring
s[i...n - 1].
A suffix array will contain integers that represent the starting
↪  indexes of the all the suffixes of a given string, after the
↪  aforementioned suffixes are sorted.

Note:  The order of the sorted cyclic shifts is equivalent to the
↪  order of the sorted suffixes

LCP (Longest Common Prefix) array:
A value lcp[SA[i]] indicates length of the longest common prefix of
↪  the suffixes indexed in SA[i] and SA[i+1].

txt[0..n-1] = "b a n a n a $"
               0 1 2 3 4 5 6

suffix[]  = {6,    5,   3,     1,        0,         4,    2}
          = {"$", "a", "ana", "anana", "banana", "na", "nana"}
            ---------------------------------------------------
             0     1    2      3        4         5     6

for (int i = 1; i < n; i++)
    cout << LCP[SA[i]] << " ";
lcp        = {0, 1, 3, 0, 0, 2, 0}
lcp[SA[0]] = LCP of "$" and "a"          = 0
lcp[SA[1]] = LCP of "a" and "ana"        = 1
lcp[SA[2]] = LCP of "ana" and "anana"    = 3
lcp[SA[3]] = LCP of "anana" and "banana" = 0
lcp[SA[4]] = LCP of "banana" and "na"    = 0
lcp[SA[5]] = LCP of "na" and "nana"      = 2
*/

void countingSort(vi &SA, vi &eq)
{
  vi count(sz(eq));

  for (int i = 0; i < sz(eq); i++)
    count[eq[i]]++;

  for (int i = 1; i < sz(eq); i++)
    count[i] += count[i - 1];

  vi out(sz(eq));

  for (int i = sz(eq) - 1; i >= 0; i--)
  {
    int val = eq[SA[i]];
    out[count[val] - 1] = SA[i];
    count[val]--;
  }
  SA = out;
}

pair<vi, vi> Suffix_Array_and_LCP(const string &s)
{
  int n = sz(s);
  vector<pair<char, int>> aux(n);

  for (int i = 0; i < n; i++)
    aux[i] = {s[i], i};

  sort(all(aux));

  vi SA(n);

  for (int i = 0; i < n; i++)
    SA[i] = aux[i].second;
```

```
vi eq(n);

eq[SA[0]] = 0;
for (int i = 1; i < n; i++)
  eq[SA[i]] = (aux[i - 1].first != aux[i].first) ? eq[SA[i - 1]] + 1
  ↪  : eq[SA[i - 1]];

for (int h = 0; (1 << h) < n; h++)
{
  int k = 1 << h;

  for (int i = 0; i < n; i++)
    SA[i] = (SA[i] - k + n) % n;

  countingSort(SA, eq);

  vi aux(n);
  aux[SA[0]] = 0;
  for (int i = 1; i < n; i++)
  {
    pair<int, int> ant = {eq[SA[i - 1]],
                          eq[(SA[i - 1] + k) % n]};
    pair<int, int> act = {eq[SA[i]], eq[(SA[i] + k) % n]};
    aux[SA[i]] = (ant != act) ? aux[SA[i - 1]] + 1 : aux[SA[i - 1]];
  }

  eq = aux;
}

vi LCP(n);

int k = 0;
for (int i = 0; i < n - 1; i++)
{
  int j = SA[eq[i] - 1];

  while (s[i + k] == s[j + k])
    k++;

  LCP[i] = k;

  k = max(k - 1, 0);
}
  return {SA, LCP};
}

int main()
{
  ios::sync_with_stdio(0);
  cin.tie(nullptr);
  string s;
  cin >> s;
  s += "$";
  pair<vi, vi> result = Suffix_Array_and_LCP(s);
  vi SA = result.first;
  vi LCP = result.second;
  return 0;
}
```

Trie

```
struct Node
{
  bool isWord = false;
  map<char, Node *> letters;
};
struct Trie
{
  Node *root;
  Trie()
  {
    root = new Node();
  }
  inline bool exists(Node *actual, const char &c)
  {
    return actual->letters.find(c) != actual->letters.end();
  }
  void InsertWord(const string &word)
  {
    Node *current = root;
    for (auto &c : word)
    {
      if (!exists(current, c))
        current->letters[c] = new Node();
      current = current->letters[c];
    }
```

```
    current->isWord = true;
  }
  bool FindWord(const string &word)
  {
    Node *current = root;
    for (auto &c : word)
    {
      if (!exists(current, c))
        return false;
      current = current->letters[c];
    }
    return current->isWord;
  }
  void printRec(Node *actual, string acum)
  {
    if (actual->isWord)
    {
      cout << acum << "\n";
    }
    for (auto &next : actual->letters)
      printRec(next.second, acum + next.first);
  }
  void printWords(const string &prefix)
  {
    Node *actual = root;
    for (auto &c : prefix)
    {
      if (!exists(actual, c))
        return;
      actual = actual->letters[c];
    }
    printRec(actual, prefix);
  }
};
```

Z function

```
/**
z[i] = longitud de la substring más grande que empieza en i
y que también es un prefijo de la string s

s = a a b x a a y a a b
z = 0 1 0 0 2 1 0 3 1 0
  -----------------------
    0 1 2 3 4 5 6 7 8 9
*/
vector<int> Z_function(const string &s)
{
  int n = s.size();
  vector<int> z(n, 0);
  int l = 0, r = 0;
  z[0] = 0;
  for (int i = 1; i < n; i++)
  {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r)
    {
      l = i;
      r = i + z[i];
    }
  }
  return z;
}
```

Dynamic Programming

Coin Combinations I

```
const int X = 1e6 + 1;
ll dp[X];
ll coin_combinations_I()
{
  int n, x; cin >> n >> x;
  if (n <= 0 || x < 0) return 0;

  vector<ll> coins(n);
  for (int i = 0; i < n; i++)
      cin >> coins[i];
  dp[0] = 1;

  for (int sum = 1; sum <= x; sum++)
    for (int c : coins)
```

```
      if (sum - c >= 0)
        {
          dp[sum] += dp[sum - c];
          dp[sum] %= MOD;
        }
  return dp[x];
}
```

Coin Combinations II

```
const int X = 1e6 + 1;
ll dp[X];
ll coin_combinations_II()
{
  int n, x; cin >> n >> x;
  if (n <= 0 || x < 0) return 0;

  vector<ll> coins(n);
  for (int i = 0; i < n; i++)
    cin >> coins[i];
  dp[0] = 1;

  for (int c : coins)
    for (int sum = 0; sum <= x; sum++)
      if (sum - c >= 0)
      {
        dp[sum] += dp[sum - c];
        dp[sum] %= MOD;
      }
  return dp[x];
}
```

Digit DP

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const int N = 19;
const int DIG = 10;
const int CEROS = 2;
const int LIBRE = 2;

ll dp[N][DIG][CEROS][LIBRE];

ll counting(string &numero, int n, int ant, bool ceros, bool libre)
{
  if (n == 0)
    return 1;

  int dig = numero[numero.size() - n] - '0';

  if (ant != -1 && dp[n][ant][ceros][libre] != -1)
    return dp[n][ant][ceros][libre];

  ll res = 0;
  int tope = (libre) ? 9 : dig;

  for (int j = 0; j <= tope; j++)
  {

    if (j == ant && !ceros)
      continue;

    res += counting(numero, n - 1, j, ceros && j == 0, libre || j <
    ↪  dig);
  }

  if (ant != -1)
    dp[n][ant][ceros][libre] = res;
  return res;
}

// Your task is to count the number of integers between a and b where
↪  no two adjacent digits are the same.
int main()
{
  ios::sync_with_stdio(0);
  cin.tie(nullptr);

  ll a, b;
  cin >> a >> b;
```

```
  a--;
  memset(dp, -1, sizeof(dp));
  string b_s = to_string(b);
  ll res = counting(b_s, b_s.size(), -1, 1, 0);

  if (a >= 0)
  {
    memset(dp, -1, sizeof(dp));
    string a_s = to_string(a);
    res -= counting(a_s, a_s.size(), -1, 1, 0);
  }

  cout << res << endl;
  return 0;
}
```

Edit Distance

```
const int N = 5000+1;
const int M = 5000+1;
int dp[N][M];
int edit_distance()
{
  string s, t; cin >> s >> t;
  int n = s.size(); int m = t.size();

  for (int j = m; j >= 0; j--)
    dp[n][j] = t.size() - j;
  for (int i = n; i >= 0; i--)
    dp[i][m] = s.size() - i;

  for (int i = n - 1; i >= 0; i--)
    for (int j = m - 1; j >= 0; j--)
      if (s[i] == t[j])
        dp[i][j] = dp[i + 1][j + 1];
      else
        dp[i][j] = min({1 + dp[i + 1][j + 1],
                        1 + dp[i + 1][j],
                        1 + dp[i][j + 1]});

  return dp[0][0];
}
```

Knapsack 01

```
const ll W = 1e5 + 1;
const ll N = 100 + 1;
ll value[N], weight[N], dp[N][W];
ll knapsack_01()
{
  int n, w; cin >> n >> w;
  for (int i = 0; i < n; i++)
  { cin >> weight[i]; cin >> value[i]; }

  for (int i = 1; i <= n; i++)
    for (int j = 0; j <= w; j++)
      if (j - weight[i - 1] < 0)
        dp[i][j] = dp[i - 1][j];
      else
        dp[i][j] = max(dp[i - 1][j],
        dp[i - 1][j - weight[i - 1]] + value[i - 1]);

  return dp[n][w];
}
```

Knapsack 01 optimized

```
const ll W = 1e5 + 1;
const ll N = 100 + 1;
ll value[N], weight[N], dp[W];
ll knapsack_01_op()
{
  ll n, w; cin >> n >> w;
  for (int i = 0; i < n; i++)
  { cin >> weight[i]; cin >> value[i]; }

  for (int i = 0; i < n; i++)
    for (int j = w; j >= 0; j--)
      if (j - weight[i] >= 0)
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

  return dp[w];
```

```
}
```

Knapsack 2

```
const int N = 100 + 1;
const int MAX_V = N * 1e3 + 1;
ll dp[N][MAX_V], value[N], weight[N];
// Es como un coin change
ll knpasack_01_2()
{
  int n, w; cin >> n >> w;
  for (int i = 0; i < n; i++)
  { cin >> weight[i]; cin >> value[i]; }

  for (int i = 0; i < N; i++)
    for (int j = 0; j < MAX_V; j++)
      dp[i][j] = INT_MAX;

  dp[0][0] = 0;
  ll res = 0;
  for (int i = 1; i <= n; i++){
    for (int j = 0; j < MAX_V; j++)
    {
      if (j - value[i - 1] < 0)
        dp[i][j] = dp[i - 1][j];
      else
        dp[i][j] = min(dp[i - 1][j],
                       dp[i - 1][j - value[i - 1]] + weight[i - 1]);
      if (dp[i][j] <= w)
        res = j;
    }
  }
  return res;
}
```

Knapsack 2 optimized

```
const int N = 100 + 1;
const int MAX_V = N * 1e3 + 1;
ll dp[MAX_V], value[N], weight[N];
ll knpasack_01_2_op()
{
  int n, w; cin >> n >> w;

  for (int i = 0; i < n; i++)
  { cin >> weight[i]; cin >> value[i]; }

  for (int i = 0; i < MAX_V; i++)
    dp[i] = INT_MAX;

  dp[0] = 0;
  ll res = 0;
  for (int i = 1; i <= n; i++)
  {
    for (int j = MAX_V - 1; j >= 0; j--)
    {
      if (j - value[i - 1] >= 0)
        dp[j] = min(dp[j], dp[j - value[i - 1]] + weight[i - 1]);
      if (dp[j] <= w)
        res = max((ll)j, res);
    }
  }
  return res;
}
```

Longest Common Subsecuence

```
int lcs(string &a, string &b)
{
  int m = a.size(), n = b.size();
  vector<vector<int>> aux(m + 1, vector<int>(n + 1));
  for (int i = 1; i <= m; ++i)
  {
    for (int j = 1; j <= n; ++j)
    {
      if (a[i - 1] == b[j - 1])
        aux[i][j] = 1 + aux[i - 1][j - 1];
      else
        aux[i][j] = max(aux[i - 1][j], aux[i][j - 1]);
    }
  }
  return aux[m][n];
}
```

Longest Increasing Subsecuence

```
int LIS(vector<int> &nums)
{
  int n = nums.size();
  vector<int> lis_aux;
  for (int i = 0; i < n; i++)
  {
    int x = nums[i];
    int pos = lower_bound(lis_aux.begin(), lis_aux.end(), x) -
    ↪  lis_aux.begin();
    if (pos == lis_aux.size())
      lis_aux.push_back(x);
    else
      lis_aux[pos] = x;
  }
  return lis_aux.size();
}
```

Geometry

Closest Pair

```
// encuentra la pareja de puntos más cercana en O(nlogn)

#include <bits/stdc++.h>
using namespace std;
#define ALL(x) x.begin(), x.end()

// se pueden cambiar para que funcione con double o float
using fl = long double;
#define HYPOT hypotl
#define ABS fabsl

struct pt;
using vpt = vector<pt>;
using ppt = pair<pt,pt>;
constexpr fl INF = numeric_limits<fl>::infinity();

struct pt { fl x, y; }; // aquí se podría poner más info extra a según
bool cmpx(const pt& a, const pt& b) { return a.x < b.x; }
bool cmpy(const pt& a, const pt& b) { return a.y < b.y; }
fl dist(const pt& a, const pt& b) { return HYPOT(a.x-b.x, a.y-b.y); }

// se usa para escoger pares
inline const pair<ppt,fl>& best_pair(const pair<ppt,fl>& a, const
↪  pair<ppt,fl>& b)
{
    return b.second < a.second ? b : a;
}

// para crear pares de puntos con su distancia
inline pair<ppt, fl> mk_p(const pt& a, const pt& b)
{
    return {{a, b}, dist(a,b)};
}

// se usa para el caso base, es O(1) en ese contexto
inline pair<ppt,fl> closest_brute(const vpt& P, int start, int end)
{
    pair<ppt,fl> p = {{P[start],P[start]}, INF};
    for (int i = start; i < end; ++i)
    for (int j = i+1; j < end; ++j)
        p = best_pair(p, mk_p(P[i], P[j]));
    return p;
}

// suena O(n^2), pero por matemagia divertida esto es realmente O(1)
↪  en el
// contexto de la función completa, y está por ahí de 7 y 12 la cota
↪  superior
inline pair<ppt,fl> strip_closest(const vpt& strip, pair<ppt,fl>
↪  closest)
{
    for (int i = 0; i < strip.size(); ++i)
    for (int j = i+1; j < strip.size() and (strip[j].y - strip[i].y) <
    ↪  closest.second; ++j)
        closest = best_pair(closest, mk_p(strip[i], strip[j]));
    return closest;
}

pair<ppt,fl> aux_closest_pair(const vpt& Px, const vpt& Py, int start,
↪  int end)
{
    // caso base O(1)
    if (end-start <= 3) return closest_brute(Px, start, end);
```

```
    int mid = start + ((end-start)>>1);
    pt midpt = Px[mid];

    // Dividimos en izq y der el ordenamiento vertical O(n)
    vpt Pyl, Pyr;
    Pyl.reserve(mid-start); Pyr.reserve(end-mid+1);
    for (const pt& p : Py)
        if (p.x <= midpt.x) Pyl.push_back(p);
        else Pyr.push_back(p);

    // divide
    auto closest = best_pair(aux_closest_pair(Px, Pyl, start, mid),
            aux_closest_pair(Px, Pyr, mid, end));

    // vencerás: la pareja es una de las mitades, o cruza la franja
    ↪ media
    vpt strip;
    for (const pt& p : Py) // O(n)
        if (ABS(p.x - midpt.x) < closest.second)
            strip.push_back(p);
    return strip_closest(strip, closest);
}

inline pair<ppt,fl> closest_pair(vpt& P) // O(nlogn) divide y vencerás
{
    vpt Px(P), Py(P);
    sort(ALL(Px), cmpx); sort(ALL(Py), cmpy);
    return aux_closest_pair(Px, Py, 0, P.size());
}
```

Complex Points

```
#include <complex>
// definimos un punto como un complejo
//typedef long long C; // no nos sirve para algunos trucos como abs y
↪ arg
typedef long double C;
typedef complex<C> P;
#define X real()
#define Y imag()
#define C_EPS 1e-12
// no importa si C es punto flotante
inline bool operator<(P a, P b) { return a.X < b.X || (a.X == b.X &&
↪ a.Y < b.Y); }
inline C cross(P a, P b) { return (conj(a)*b).Y; }
inline C dir(P u, P v, P w) { return cross(v-u,w-v); }
inline bool ccw(P u, P v, P w) { return dir(u,v,w) > C_EPS; }
inline bool cw(P u, P v, P w) { return dir(u,v,w) < -C_EPS; }
inline bool colin(P u, P v, P w)
{ C d = dir(u,v,w); return -C_EPS <= d && d <= C_EPS; }
int seg_intersect(P a, P b, P c, P d)
{ // 0.ab no toca cd.
  // 1.se tocan solo en un extremo.
  // 2.son colineales y se tocan infinitas veces.
  // 3.se tocan al interior de alguno exactamente una vez.
    if (colin(a,b,c) && colin(a,b,d))
    {
        if (c > d) swap(c,d);
        if (a > b) swap(a,b);
        if (abs(b-c) <= C_EPS || abs(d-a) <= C_EPS) return 1;
        if (a < c && c < b) return 2;
        if (c < a && a < d) return 2;
        return 0;
    }
    if (abs(a-c)<=C_EPS || abs(a-d)<=C_EPS || abs(b-c)<=C_EPS ||
    ↪ abs(b-d)<=C_EPS)
        return 1;
    if (dir(a,b,c)*dir(a,b,d) < 0 && dir(c,d,a)*dir(c,d,b) < 0)
        return 3;
    return 0;
}
// solo si C es punto flotante
inline C dist(P a, P b) { return abs(b-a); }
inline C dist_pt_line(P l1, P l2, P p) { return
↪ cross(l1-p,l2-p)/dist(l1,l2); }
inline C angle_xaxis(P p) { return arg(p); }
inline P polr(C mag, C angle) { return polar(mag,angle); }
inline P rotated(P v, C angle) { return v*polar(1,angle); }

int main()
{
    P p = {3,-4};
    cout << "p: " << p.X << ' ' << p.Y << endl; // mostramos parte X y
    ↪ Y
```

```
    return 0;
}
```

Convex Hull

```
// algoritmo de graham
using ff = float;

// -1 reloj (derecho), +1 contrarreloj (izquierdo), 0 colineal
int giro(v2 a, v2 b, v2 c) {
    ff v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1;
    if (v > 0) return +1;
    return 0;
}

// regresa lista con el cierre convexo (CH)
vector<v2> cvxhull(vector<v2>& pts) {
    // caso trivial
    if (pts.size() <= 2) return pts;

    // punto mas chico en Y
    v2 p0 = *min_element(pts.begin(), pts.end(), [](const v2& a, const
    ↪  v2& b) {
        return make_pair(a.y, a.x) < make_pair(b.y, b.x);
    });
    // ord angular con respecto a p0
    sort(pts.begin(), pts.end(), [&p0](const v2& a, const v2& b){
        int g = giro(p0, a, b);
        if (g == 0) {
            // si son colineales, escoge primero el más cercano
            return (p0-a).sqrmag() < (p0-b).sqrmag();
        }
        return g > 0;
    });

    // ordenamos últimos colineales por distancia inversa para evitar
    ↪  que 'vaya y regrese'
    ll i = (ll)pts.size() - 1;
    while (i >= 0 && giro(p0, pts[i], pts.back()) == 0) i--;
    // invertimos los últimos puntos colineales para que aparezcan
    ↪  primero los lejanos
    reverse(pts.begin()+i+1, pts.end());

    vector<v2> cvx; // la pila con el CH
    for (ll j = 0; j < (ll)pts.size(); j++) {
        // eliminamos los que induzcan un giro inválido
        while (cvx.size() > 1 && giro(cvx[cvx.size()-2], cvx.back(),
        ↪  pts[j]) < 0)
            cvx.pop_back();
        cvx.push_back(pts[j]);
    }

    return cvx;
}
```

Delaunay

```
//// Point.h

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

/**
 * Author: Ulf Lundstrom
 * Date: 2009-02-26
 * License: CC0
 * Source: My head with inspiration from tinyKACTL
 * Description: Class to handle points in the plane.
 *     T can be e.g. double or long long. (Avoid int.)
 * Status: Works fine, used a lot
 */
#pragma once

//template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
```

```
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    // T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    //double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    //double angle() const { return atan2(y, x); }
    //P unit() const { return *this/dist(); } // makes dist()=1
    //P perp() const { return P(-y, x); } // rotates +90 degrees
    //P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    //P rotate(double a) const {
        //return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    //friend ostream& operator<<(ostream& os, P p) {
        //return os << "(" << p.x << "," << p.y << ")"; }
};

//// Point.h


//// FastDelaunay.h

/**
 * Author: Philippe Legault
 * Date: 2016
 * License: MIT
 * Source: https://github.com/Bathlamos/delaunay-triangulation/
 * Description: Fast Delaunay triangulation.
 * Each circumcircle contains none of the input points.
 * There must be no duplicate points.
 * If all points are on a line, no triangles will be returned.
 * Should work for doubles as well, though there may be precision
 ↪  issues in 'circ'.
 * Returns triangles in order \{t[0][0], t[0][1], t[0][2], t[1][0],
 ↪  \dots\}, all counter-clockwise.
 * Time: O(n \log n)
 * Status: stress-tested
 */
#pragma once
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    lll p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }
```

```
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
           (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
        while (circ(e->dir->F(), H(base), e->F())) { \
            Q t = e->dir; \
            splice(e, e->prev()); \
            splice(e->r(), e->r()->prev()); \
            e->o = H; H = e; e = t; \
        }
    for (;;) {
        DEL(LC, base->r(), o);  DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
    sort(all(pts));  assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
    return pts;
}
```

```
//// FastDelaunay.h
```

KD Tree

```
#include <bits/stdc++.h>
using namespace std;

using Coord = double; // tipo de coordenadas
constexpr Coord EPS = 1e-9;
const int MAX_D = 5; // numero de dimensiones máximo

struct KDTree {
    struct Point {
        Coord x[MAX_D];
        int id;
    };
    struct Node {
        Point p;
        int dim;
        Node *l, *r;
    };
    int D;
    Node* root;

    KDTree(int _D) : D(_D), root(nullptr) {}

    Node* build(vector<Point>& pts, int l, int r, int depth) {
        if (l >= r) return nullptr;
        int dim = depth % D;
        int m = (l + r) / 2;
        nth_element(pts.begin()+l, pts.begin()+m, pts.begin()+r,
            [dim](const Point& a, const Point& b){ return a.x[dim] <
            ↪  b.x[dim]; });
        Node* node = new Node{pts[m], dim, nullptr, nullptr};
        node->l = build(pts, l, m, depth+1);
        node->r = build(pts, m+1, r, depth+1);
        return node;
    }

    void build(vector<Point>& pts) { root = build(pts, 0, pts.size(),
    ↪  0); }
```

```
// ---- Distance function ----
Coord dist(const Point& a, const Point& b) {
#ifdef MANHATTAN
    Coord d = 0;
    for (int i=0;i<D;i++) d += abs(a.x[i]-b.x[i]);
    return d;
#else
    Coord d = 0;
    for (int i=0;i<D;i++){ Coord diff=a.x[i]-b.x[i]; d +=
    ↪ diff*diff; }
    return d;
#endif
}

// ---- Nearest neighbor ----
void nearest(Node* node, const Point& target, Point& best, Coord&
↪ bestd) {
    if (!node) return;
    Coord d = dist(node->p, target);
#ifdef MANHATTAN
    if (d + EPS < bestd) { bestd = d; best = node->p; }
#else
    if (d < bestd - EPS) { bestd = d; best = node->p; }
#endif
    int dim = node->dim;
    Node *first = target.x[dim] < node->p.x[dim] ? node->l :
    ↪ node->r;
    Node *second = first == node->l ? node->r : node->l;
    nearest(first, target, best, bestd);
    Coord diff = abs(target.x[dim] - node->p.x[dim]);
#ifdef MANHATTAN
    if (diff < bestd - EPS) nearest(second, target, best, bestd);
#else
    if (diff*diff < bestd - EPS) nearest(second, target, best,
    ↪ bestd);
#endif
}

Point nearest(const Point& target) {
    Point best; Coord bestd = numeric_limits<Coord>::max();
    nearest(root, target, best, bestd);
    return best;
}

    void range_query(Node* node, const Coord low[], const Coord
    ↪ high[], vector<Point>& out) {
        if (!node) return;
        const auto& P = node->p;
        bool inside = true;
        for (int i=0; i<D; i++) {
            if (P.x[i] < low[i] - EPS || P.x[i] > high[i] + EPS)
            { inside = false; break; }
        }
        if (inside) out.push_back(P);

        int dim = node->dim;
        if (low[dim] - EPS <= P.x[dim]) range_query(node->l, low,
        ↪ high, out);
        if (high[dim] + EPS >= P.x[dim]) range_query(node->r, low,
        ↪ high, out);
    }

    vector<Point> range_query(const vector<Coord>& low, const
    ↪ vector<Coord>& high) {
        vector<Point> res;
        Coord lo[MAX_D], hi[MAX_D];
        for (int i=0;i<D;i++){ lo[i] = low[i]; hi[i] = high[i]; }
        range_query(root, lo, hi, res);
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, D; cin >> n >> D; // n - #pts, D - #dims
    vector<KDTree::Point> pts(n);
    for (int i=0;i<n;i++) {
        for (int j=0;j<D;j++) cin >> pts[i].x[j];
        pts[i].id = i;
    }
    KDTree tree(D);
    tree.build(pts);

    // Example nearest neighbor query
    KDTree::Point q;
```

```
    for (int j=0;j<D;j++) cin >> q.x[j];
    auto nearest_pt = tree.nearest(q);
    cout << "Nearest id: " << nearest_pt.id << "\n";

    // Example range query
    vector<Coord> low(D), high(D);
    for (int j=0;j<D;j++) cin >> low[j];
    for (int j=0;j<D;j++) cin >> high[j];
    auto ans = tree.range_query(low, high);
    for (auto &p : ans) {
        cout << p.id;
        for (int j=0;j<D;j++) cout << " " << p.x[j];
        cout << "\n";
    }
}
```

Li Chao Tree

```
// O(nlogC) con C el máximo |x_i|
// Para funciones que se intersectan a lo más una vez, nos da el min de
// todas ellas en tiempo logaritmico (por ejemplo para encontrar el CH
↪  de
// varias rectas)
typedef long long C; // esto se puede cambiar
typedef complex<C> P;
#define X real()
#define Y imag()

C dot(P a, P b) {
    return (conj(a) * b).X;
}

// rectas por ejemplo, pero puede ser otra fn
C f(P a,  C x) {
    return dot(a, {x, 1});
}

const int maxn = 2e5; // max número de funciones

P line[4 * maxn]; // segment tree del arreglo de lineas

void add_line(P nw, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    bool lef = f(nw, l) < f(line[v], l);
    bool mid = f(nw, m) < f(line[v], m);
    if(mid) {
        swap(line[v], nw);
    }
    if(r - l == 1) {
        return;
    } else if(lef != mid) {
        add_line(nw, 2 * v, l, m);
    } else {
        add_line(nw, 2 * v + 1, m, r);
    }
}

C get(int x, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    if(r - l == 1) {
        return f(line[v], x);
    } else if(x < m) {
        return min(f(line[v], x), get(x, 2 * v, l, m));
    } else {
        return min(f(line[v], x), get(x, 2 * v + 1, m, r));
    }
}

// para agregar líneas: add_line({m, b})
// para obtener el mínimo del arreglo de líneas en x: get_line(x)
```

Line

```
// linea 2d

using ff = float;

// distancia linea-punto, donde la línea se define por a->b
ff dist_lin_v2(v2 p, v2 a, v2 b) {
    ff A = cross(b-a, p-a);
    return fabs(A)/((b-a).norm());
}

// si se intersectan dos líneas (p->r y q->s), y en out dónde
```

```
bool intersect_lin_lin(v2 p, v2 r, v2 q, v2 s, v2 &out){
  double rxs = cross(r, s);
  if (fabs(rxs) < 1e-12)
    return false;
  double t = cross(q - p, s) / rxs;
  out = { p.x + t * r.x,
          p.y + t * r.y };
  return true;
}
```

Minimum Enclosing Circle

```
// Tienes que definir EPS

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

/// Points.h

/**
 * Author: Ulf Lundstrom
 * Date: 2009-02-26
 * License: CC0
 * Source: My head with inspiration from tinyKACTL
 * Description: Class to handle points in the plane.
 *     T can be e.g. double or long long. (Avoid int.)
 * Status: Works fine, used a lot
 */
#pragma once

//template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    //T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    //double angle() const { return atan2(y, x); }
    //P unit() const { return *this/dist(); } // makes dist()=1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    //P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    //P rotate(double a) const {
        //return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    //friend ostream& operator<<(ostream& os, P p) {
        //return os << "(" << p.x << "," << p.y << ")"; }
};

/// circumcircle.h

/**
 * Author: Ulf Lundstrom
 * Date: 2009-04-11
 * License: CC0
 * Source: http://en.wikipedia.org/wiki/Circumcircle
 * Description:\\
\begin{minipage}{75mm}
    The circumcirle of a triangle is the circle intersecting all three
    vertices. ccRadius returns the radius of the circle going through
    ↪ points A,
    B and C and ccCenter returns the center of the same circle.
\end{minipage}
\begin{minipage}{15mm}
\vspace{-2mm}
\includegraphics[width=\textwidth]{content/geometry/circumcircle}
\end{minipage}
 * Status: tested
 */
// Circumcírculo

//typedef Point<double> P;
//double ccRadius(const P& A, const P& B, const P& C) {
    //return (B-A).dist()*(C-B).dist()*(A-C).dist()/
            //abs((B-A).cross(C-A))/2;
//}
```

```
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}

// MinimumEnclosingCircle.h

/**
 * Author: Andrew He, chilli
 * Date: 2019-05-07
 * License: CC0
 * Source: folklore
 * Description: Computes the minimum circle that encloses a set of
 ↪  points.
 * Time: expected O(n)
 * Status: stress-tested
 */
// Este es el algoritmo de Welzl, no el de Megiddo (O(n) determinista)
pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

Online Convex Hull

```
// O(logi) cada inserción, O(logi) consulta, O(nlogn) construcción
↪  total
struct pt
{
    ll x, y;
    pt(ll x_ = 0, ll y_ = 0) : x(x_), y(y_) {}
    bool operator<(const pt p) const // ord lexicográfico
    {
        if (x != p.x) return x < p.x;
        return y < p.y;
    }
    inline bool operator==(const pt p) const { return x == p.x and y
    ↪  == p.y; }
    inline pt operator+(const pt p) const { return pt(x+p.x, y+p.y); }
    inline pt operator-(const pt p) const { return pt(x-p.x, y-p.y); }
    inline pt operator*(const ll c) const { return pt(c*x, c*y); }
    inline ll operator*(const pt p) const { return x*p.x + y*p.y; } //
    ↪  producto interior
    inline int operator^(const pt p) const // signo del producto
    ↪  exterior
    {
        // NOTE: esto debe modificarse a según. estos productos
        ↪  fácilmente
        //       podrían ser muy grandes, por lo que se podría quizás
        ↪  adaptar
        //       como...
        //   --> long double l = x*(long double)p.y; // y análogamente
        ↪  para r
        //   --> long long l = x*p.y; // y análogamente para r
        //   --> dejarlo así y definir i128
        // x*p.y - y*p.x
        i128 l = mul_ll(x, p.y);
        i128 r = mul_ll(y, p.x);
        if (l == r) return 0;
        if (l < r) return -1;
        return 1;
    }
    friend istream& operator>>(istream& in, pt& p)
    {
        return in >> p.x >> p.y;
    }
};

bool ccw(pt p, pt q, pt r)
{
    return ((q-p)^(r-q)) > 0;
}
```

```cpp
struct upper
{
    set<pt> se;
    set<pt>::iterator it;

    int is_under(pt p) // 1 -> in ; 2 -> border
    {
        it = se.lower_bound(p);
        if (it == se.end()) return 0;
        if (it == se.begin()) return p == *it ? 2 : 0;
        if (ccw(p, *it, *prev(it))) return 1;
        return ccw(p, *prev(it), *it) ? 0 : 2;
    }
    void insert(pt p)
    {
        if (is_under(p)) return;
        if (it != se.end()) while (next(it) != se.end() and
        ↪  !ccw(*next(it), *it, p))
            it = se.erase(it);
        if (it != se.begin()) while (--it != se.begin() and !ccw(p,
        ↪  *it, *prev(it)))
            it = se.erase(it);
        se.insert(p);
    }
};

struct dyn_hull
{
    upper U,L;

    int is_inside(pt p) // 1 -> in ; 2 -> border
    {
        int u = U.is_under(p), l = L.is_under({-p.x, -p.y});
        if (!u or !l) return 0;
        return max(u,l);
    }
    void insert(pt p)
    {
        U.insert(p);
        L.insert({-p.x, -p.y});
    }
    int size()
    {
        int ans = U.se.size() + L.se.size();
        return ans <= 2 ? ans/2 : ans-2;
    }
};
```

Picks Theorem

```cpp
// Si tenemos un polígono con coordenadas enteras y conocemos el
↪  número de
// puntos (con coordenadas enteras) internos y en la frontera, podemos
↪  obtener
// su área en O(1):
int pick_area(int internos, int en_frontera)
{
    return internos + en_frontera/2 - 1;
}

// Podemos obtener los puntos con coordenadas enteras en la frontera
↪  en O(n):
int boundary_points(const vector<pair<int, int>>& P) {
    int n = P.size();
    int B = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        int dx = abs(P[i].first - P[j].first);
        int dy = abs(P[i].second - P[j].second);
        B += __gcd(dx, dy); // tecnicamente O(log(min(dx,dy)))
    }
    return B;
}

// Si conocemos el área y los puntos en la frontera, podemos obtener
↪  internos.
// Tomamos 2 veces el área porque podemos obtenerla sin floats con
↪  shoelace
int pick_internos(int area2x, int en_frontera)
{
    return (area2x - en_frontera + 2)/2;
}

// asi...
```

```
ll shoelace_area2x(const vector<pair<int,int>>& P)
{
    int n = P.size();
    ll area2x = 0;
    for (int i = 0; i < n; ++i)
    {
        int j = (i+1)%n;
        area2x += 1LL * P[i].first * P[j].second;
        area2x -= 1LL * P[j].first * P[i].second;
    }
    return abs(area2x);
}
```

Points

```
// vector 2d

using ff = float;

struct v2 {
    ff x, y;
    v2(ff x, ff y): x(x), y(y) {}
    bool operator==(v2 const& t) const {
        return x == t.x && y == t.y;
    }
    v2& operator+=(const v2 &t) {
        x += t.x;
        y += t.y;
        return *this;
    }
    v2& operator-=(const v2 &t) {
        x -= t.x;
        y -= t.y;
        return *this;
    }
    v2& operator*=(ff t) {
        x *= t;
        y *= t;
        return *this;
    }
    v2& operator/=(ff t) {
        x /= t;
        y /= t;
        return *this;
    }
    v2 operator+(const v2 &t) const {
        return v2(*this) += t;
    }
    v2 operator-(const v2 &t) const {
        return v2(*this) -= t;
    }
    v2 operator*(ff t) const {
        return v2(*this) *= t;
    }
    v2 operator/(ff t) const {
        return v2(*this) /= t;
    }
    ff mag() const {
        return sqrt(x*x + y*y);
    }
    ff sqrmag() const {
        return x*x + y*y;
    }
};

v2 operator*(ff a, v2 b) {
    return b * a;
}

inline ff dot(v2 a, v2 b) {
    return a.x * b.x + a.y * b.y;
}

inline ff cross(v2 a, v2 b) {
    return a.x*b.y - a.y*b.x;
}
```

Shoelace Formula

```
// Obtiene en O(n) el área de un polígono simple con vértices en orden
// horario o anti-horario.
double shoelaceArea(const vector<pair<int, int>>& P) {
    int n = P.size();
    long long area = 0;
```

```
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        area += 1LL * P[i].first * P[j].second;
        area -= 1LL * P[j].first * P[i].second;
    }
    return abs(area) / 2.0;
}
```

Others

All Submask

```
// O(3^n)
int dp[1 << 16];
int cost[1 << 16];
int main()
{
  int N = 16;
  for (int mask = 0; mask < N; mask++)
  {
    for (int submask = mask; submask; submask = (submask - 1) & mask)
    {
      dp[mask] = max(dp[mask], cost[submask] + dp[mask ^ submask]);
    }
  }
  return 0;
}
```

Binpow

```
ll binpow(ll a, ll b) {
  ll res = 1;
  while (b > 0) {
    if (b & 1)
      res = res * a;
    a = a * a;
    b >>= 1;
  }
  return res;
}
```

Coordinate Compression

```
vector<int> coordinate_compression(vector<int>& coordinates){
  vector<int> compression(coordinates.size());
  vector<int> aux = coordinates;
  sort(aux.begin(), aux.end());
  aux.resize(unique(aux.begin(), aux.end()) - aux.begin());
  for (int i = 0; i < n; ++i){
    compression[i] = lower_bound(aux.begin(), aux.end(),
    ↪  coordinates[i])
    - aux.begin();
  }
  return compression;
}
```

Cover Points By Intervals

```
// Número de puntos x en 0 <= l <= r <= 1e18 tal que el
// número de intervalos que lo cubren es igual a k, con k en [1...n]
void cover_points()
{
  int n; cin >> n;
  vector<pll> pairs(n); vector<ll> val;
  for (int i = 0; i < n; i++)
  {
    ll l, r;
    cin >> l >> r;
    pairs[i] = make_pair(l, r + 1);
    val.push_back(l);
    val.push_back(r + 1);
  }

  sort(all(val));
  val.resize(unique(all(val)) - val.begin());
  vector<ll> prefix(2 * n + 1);

  for (auto p : pairs)
  {
    int l = lower_bound(all(val), p.first) - val.begin();
    int r = lower_bound(all(val), p.second) - val.begin();
    prefix[l] += 1;
```

```
    prefix[r] -= 1;
  }
  for (int i = 1; i <= 2 * n; i++)
    prefix[i] += prefix[i - 1];

  vector<ll> res(n + 1);
  for (int i = 1; i < val.size(); i++)
  {
    ll num_points = val[i] - val[i - 1];
    res[prefix[i - 1]] += num_points;
  }
  // res[i] = El número de puntos que son cubiertos por i intervalos
}
```

FFT

```
using C = double;
using CX = complex<C>;
const C PI = acos(-1);

// FFT + inverse FFT
void fft(vector<CX> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        C ang = 2 * PI / len * (invert ? -1 : 1);
        CX wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            CX w(1);
            for (int j = 0; j < len / 2; j++) {
                CX u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (CX & x : a)
            x /= n;
    }
}

// multiplicación de polinomios en O(nlogn) [recibe coefs enteros]
vector<int> polyn_multiply(vector<int> const& a, vector<int> const& b)
↪ {
    vector<CX> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

// de cuántas maneras se puede escojer (i,j) tal que a[i]+b[j]=k?
int count_sum(vector<int> const& a, vector<int> const& b, int k)
{
    vector<int> pa = ..., pb = ...;
                        // aquí guardamos entradas de 0 (o min spg)
                        ↪  hasta el
                        // valor max de a y b, donde en cada casilla
                        ↪  arr[i]
                        // llenamos el número de ocurrencias de i.
    // si tenemos a=[1,2,3], b=[2,4], construimos
    //      pa = 0x^0 + 1x^1 + 1x^2 + 1x^3 + 0x^4
```

```
    //      pb = 0x^0 + 0x^1 + 1x^2 + 0x^3 + 1x^4
    // donde el coef. de x^i es el número de apariencias de i
    r = polyn_multiply(pa, pb);
    // el resultado es el coeficiente de x^k
    return r[k];
}

// dado vectores a y b de dimension n, encuentra el producto punto de
↪  a con
// cada shift circular de b.
vector<int> all_dot_shifts(vector<int> const& a, vector<int> const& b,
↪  int n)
{
    vector<int> fa(2*n), fb(2*n);
    fa = reverse(a) + [0]*n; // pseudocódigo pythonesco
    fb = b + b; // pseudocódigo pythonesco
    r = polyn_multiply(fa,fb);

    // así, r[k] nos da el producto punto de a con el shift (k-(n-1))
    ↪  de b
}

// string matching con wildcards
vector<bool> str_match(const string& txt, const string& pat, char
↪  wildcard='*')
{
    int n = txt.size(), m = pat.size();
    int wilds = 0;
    for (char c : pat) if (c == wildcard) wilds++;
    vector<CX> a(n), b(m,0);
    for (int i = 0; i < n; i++)
    {
        C t = (2*PI*(txt[i]-'a'))/26;
        a[i] = {cos(t), sin(t)};
    }
    for (int i = 0; i < m; i++)
    {
        int j = m-i-1;
        if (pat[j] == wildcard) continue;
        C t = (2*PI*(pat[j]-'a'))/26;
        b[i] = {cos(t), sin(t)};
    }
    ab = polyn_multiply(a, b);
    vector<bool> r(n);
    for (int i = 0; i < n; i++) r[i] = ab[i] == (m - wilds);
    return r; // si r[i]=true entonces hay un match en la pos i
}
```

i128

```
// si no esta disponible
// typedef __int128_t lll;

// algunas operaciones que simulan int de 128 bits
using u64 = unsigned long long;

struct u128 { u64 hi, lo; };

// multiplica dos unsigned long long sin desborde
u128 mul_u64(u64 a, u64 b) {
    constexpr u64 MASK32 = 0xFFFFFFFF;

    u64 a_lo = a & MASK32;
    u64 a_hi = a >> 32;
    u64 b_lo = b & MASK32;
    u64 b_hi = b >> 32;

    u64 lo_lo = a_lo * b_lo;
    u64 hi_lo = a_hi * b_lo;
    u64 lo_hi = a_lo * b_hi;
    u64 hi_hi = a_hi * b_hi;

    u64 cross = hi_lo + lo_hi;
    bool carry1 = cross < hi_lo;

    u64 cross_lo = cross << 32;
    u64 lo = lo_lo + cross_lo;
    bool carry2 = lo < lo_lo;

    u64 hi = hi_hi + (cross >> 32) + carry1 + carry2;

    return { hi, lo };
}

// int de 128 bits con signo
struct i128
```

```
{
    u64 hi, lo;
    bool neg;
    inline bool operator==(const i128& o) const
    {
        return hi == o.hi and lo == o.lo and (neg == o.neg or (hi == 0
        ↪  and lo == 0));
    }
    bool operator<(const i128& o) const
    {
        if (*this == o) return false;
        if (hi == 0 and lo == 0) return !o.neg;
        if (o.hi == 0 and o.lo == 0) return neg;
        if (neg != o.neg) return neg;
        if (hi < o.hi) return !neg;
        if (hi > o.hi) return neg;
        if (lo < o.lo) return !neg;
        if (lo > o.lo) return neg;
        return false;
    }
};

// multiplica dos long long sin desborde
i128 mul_ll(ll a, ll b)
{
    bool neg = (a<0) ^ (b<0);
    u128 res = mul_u64(llabs(a), llabs(b));
    return { res.hi, res.lo, neg };
}
```

Kadanes

```
// Given an array of numbers a[0...n] find the subarray
// a[l...r] with the maximum sum
int ans = a[0], ans_l = 0, ans_r = 0;
int sum = 0, minus_pos = -1;
for (int r = 0; r < n; ++r) {
  sum += a[r];
  if (sum > ans) {
    ans = sum;
    ans_l = minus_pos + 1;
    ans_r = r;
  }
  if (sum < 0) {
    sum = 0;
    minus_pos = r;
  }
}
```

Matrix

```
template <typename T>
void matmul(vector<vector<T>> &a, vector<vector<T>> b)
{
    int n = a.size(), m = a[0].size(), p = b[0].size();
    assert(m == b.size());
    vector<vector<T>> c(n, vector<T>(p));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < p; j++)
            for (int k = 0; k < m; k++)
                // Mult with mod define
                c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % MOD;

    a = c;
}

template <typename T>
struct Matrix
{
    vector<vector<T>> mat;
    Matrix() {}
    Matrix(vector<vector<T>> a) { mat = a; }
    Matrix(int n, int m)
    {
        mat.resize(n);
        for (int i = 0; i < n; i++)
            mat[i].resize(m);
    }
    int rows() const { return mat.size(); }
    int cols() const { return mat[0].size(); }

    vector<T> &operator[](int i) { return mat[i]; }
    const vector<T> &operator[](int i) const { return mat[i]; }
```

```
    void makeiden()
    {
        for (int i = 0; i < rows(); i++)
            mat[i][i] = 1;
    }
    Matrix operator*=(const Matrix &b)
    {
        matmul(mat, b.mat);
        return *this;
    }
    Matrix operator*(const Matrix &b) { return Matrix(*this) *= b; }
    void print() const
    {
        for (int i = 0; i < rows(); i++)
        {
            for (int j = 0; j < cols(); j++)
            {
                cout << mat[i][j] << ' ';
            }
            cout << '\n';
        }
    }
};
```

Matrix Exponentiation

```
// Uso: Matrix<T> M = matPow<T>(A, n - 1);
template <typename T>
Matrix<T> matPow(Matrix<T> &base, ll p)
{
  Matrix<T> ans(base.rows(), base.cols());
  ans.makeiden();
  while (p)
  {
    if (p & 1)
      ans *= base;

    base *= base;
    p >>= 1;
  }
  return ans;
}
```

Matrix with array

```
#include <iostream>
#include <algorithm>
using namespace std;

const long long MOD = 1000000007;
// Multiplicación de matrices 2x2: C = A * B
void mult2x2(const long long A[2][2], const long long B[2][2], long
↪  long C[2][2]) {
    long long temp00 = (A[0][0] * B[0][0] + A[0][1] * B[1][0]) % MOD;
    long long temp01 = (A[0][0] * B[0][1] + A[0][1] * B[1][1]) % MOD;
    long long temp10 = (A[1][0] * B[0][0] + A[1][1] * B[1][0]) % MOD;
    long long temp11 = (A[1][0] * B[0][1] + A[1][1] * B[1][1]) % MOD;
    C[0][0] = temp00;
    C[0][1] = temp01;
    C[1][0] = temp10;
    C[1][1] = temp11;
}
// Exponenciación binaria de la matriz 2x2 M a la potencia exp. El
↪  resultado se almacena en 'result'.
void matPow2x2(const long long M[2][2], long long exp, long long
↪  result[2][2]) {
    // Inicializamos 'result' como la identidad.
    result[0][0] = 1; result[0][1] = 0;
    result[1][0] = 0; result[1][1] = 1;

    long long base[2][2] = { {M[0][0], M[0][1]}, {M[1][0], M[1][1]} };

    while(exp > 0) {
        if(exp & 1) {
            long long temp[2][2];
            mult2x2(result, base, temp);
            result[0][0] = temp[0][0]; result[0][1] = temp[0][1];
            result[1][0] = temp[1][0]; result[1][1] = temp[1][1];
        }
        long long temp[2][2];
        mult2x2(base, base, temp);
        base[0][0] = temp[0][0]; base[0][1] = temp[0][1];
        base[1][0] = temp[1][0]; base[1][1] = temp[1][1];
        exp >>= 1;
```

```
    }
}

// s_0 y s_1 son los casos base y n es el número de pasos a avanzar a
↪  partir de s1.
long long fib(long long n, long long s_0, long long s_1) {
    // Si n es 0, la matriz identidad deja la columna [s_1, s_0] y se
    ↪  devuelve s_1.
    if(n == 0) return s_1 % MOD;
    long long M[2][2] = { {1, 1}, {1, 0} };
    long long power[2][2];
    matPow2x2(M, n, power);
    // Multiplicamos la matriz resultante por la columna [s_1, s_0]:
    long long res = (power[0][0] * s_1 + power[0][1] * s_0) % MOD;
    return res;
}
```

Modular binpow

```
ll modpow(ll a, ll b, ll m) {
  a %= m;
  ll res = 1;
  while (b > 0) {
    if (b & 1)
      res = res * a % m;
    a = a * a % m;
    b >>= 1;
  }
  return res;
}
```

Preifix Sum 2D

```
// Se le suma uno MAX_SIDE para trabajar con 1-indexed y
// que la primera fila y columnas estén llenas de 0 y no haya problema
// al acceder a incides anteriores
constexpr int MAX_SIDE = 1000;
int mtxPrefix[MAX_SIDE + 1][MAX_SIDE + 1];
int mtx[MAX_SIDE + 1][MAX_SIDE + 1];

int main()
{
  int N, Q;
  cin >> N >> Q;

  for(int i = 1; i<= N; i++)
    for(int j = 1; j <= N; j++)
      cin >> mtx[i][j];

  // Construye la prefix matrix
  for (int i = 1; i <= N; i++)
    for (int j = 1; j <= N; j++)
      mtxPrefix[i][j] = mtx[i][j] + mtxPrefix[i - 1][j]
                        + mtxPrefix[i][j - 1] - mtxPrefix[i - 1][j -
                        ↪  1];

  // Responder a querys donde te piden la suma de la matriz con esquina
  // superior izquierda (from_row, from_col) y esquina inferior derecha
  // (to_row, to_col), en este caso 1-indexed
  while (Q--)
  {
    int from_row, to_row, from_col, to_col;
    cin >> from_row >> from_col >> to_row >> to_col;
    ll total_sum = mtxPrefix[to_row][to_col] - mtxPrefix[from_row -
    ↪  1][to_col] -
                   mtxPrefix[to_row][from_col - 1] +
                   mtxPrefix[from_row - 1][from_col - 1];
    cout << total_sum << endl;
  }
}
```

Static Range Minimum Query

```
// O(n) construccion, O(1) query
template<typename T> struct rmq
{
    vector<T> v;
    ll n; static const int b = 62; // cambia a 30 si usamos int en
    ↪  lugar de ll
    vector<ll> mask, t;

    // la operación se puede cambiar si el resultado es elemento del
    ↪  cjto
```

```
    ll op(ll x, ll y) { return v[x] < v[y] ? x : y; }
    ll msb(ll x) { return __builtin_clz(1)-__builtin_clz(x); }
    ll small (ll r, ll sz = b) { return r-msb(mask[r]&((1<<sz)-1)); }
    rmq(const vector<T>& v_) : v(v_), n(v.size()), mask(n), t(n)
    {
        for (ll i = 0, at = 0; i < n; mask[i++] = at |= 1)
        {
            at = (at<<1)&((1ll<<b)-1);
            while (at and op(i, i-msb(at&-at)) == i)
                at ^= at&-at;
        }
        for (ll i = 0; i < n/b; i++)
            t[i] = small(b*i+b-1);
        for (ll j = 1; (1ll<<j) <= n/b; j++)
        for (ll i = 0; i+(1ll<<j) <= n/b; i++)
                t[n/b*j+1] = op(t[n/b*(j-1)+i],
                ↪  t[n/b*(j-1)+i+(1ll<<(j-1))]);
    }
    T query(ll l, ll r)
    {
        if (r-l+1 <= b) return v[small(r,r-l+1)];
        ll ans = op(small(l+b-1), small(r));
        ll x = l/b+1, y = r/b-1;
        if (x <= y)
        {
            ll j = msb(y-x+1);
            ans = op(ans, op(t[n/b*j+x], t[n/b*j+y-(1ll<<j)+1]));
        }
        return v[ans];
    }
};
```

Ternary search

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
const double PI = 3.14159265358979323846;

double f(double x) {
    return -(x - 2) * (x - 2) + PI;
}

double busqueda_ternaria_max(double l, double r, double epsilon =
↪  1e-7) {
    while (r - l > epsilon) {
        double tercio1 = l + (r - l) / 3;
        double tercio2 = r - (r - l) / 3;
        if (f(tercio1) < f(tercio2))
            l = tercio1;  // El máximo está en [tercio1, r]
        else
            r = tercio2;  // El máximo está en [l, tercio2]
    }
    return (l + r) / 2; // Aproximación del máximo
}

int main() {
    double l = 0, r = 4;
    double max_x = busqueda_ternaria_max(l, r);
    cout << fixed << setprecision(7);
    cout << "Máximo en x   " << max_x << ", f(x)   " << f(max_x) <<
    ↪  endl;
    return 0;
}
```